

UNIVERSITÉ [NOM_UNIVERSITE]

MASTER 1 – [NOM_MENTION]

Travail d'Étude et de Recherche (TER)

Systeme Multi-Agents

pour le recrutement assisté par IA

Une étude appliquée des paradigmes SMA et Agentic AI

Auteurs

Anouar Soufyani

[NOM_2]

[NOM_3]

Encadrement

[NOM_ENCADRANT]

Année universitaire 2025–2026

Table des matières

1	Présentation du projet	1
1.1	Contexte du travail	1
1.2	Problématique fonctionnelle	1
1.3	Objectifs du TER	2
1.4	Démarche et plan du rapport	2
2	État de l’art	5
2.1	Systèmes Multi-Agents (SMA) classiques	5
2.1.1	Définition et composants fondamentaux	5
2.1.2	Propriétés des agents	5
2.1.3	Architectures organisationnelles et cognitives	6
2.1.4	Coordination, négociation et protocoles formels	6
2.1.5	Apprentissage : du SMA au MARL	7
2.1.6	Émergence et intelligence collective	7
2.1.7	Applications	7
2.2	Agentic AI : définition et fondements	8
2.2.1	De l’agent à l’agenticité	8
2.2.2	Double paradigme : symbolique et neuronal	8
2.2.3	Architecture générale d’un système agentique	8
2.2.4	Du single-agent aux systèmes multi-agents agentiques	9
2.2.5	Applications structurantes	9
2.2.6	Enjeux transversaux	10
2.2.7	Trajectoire évolutive	10
2.3	Analyse comparative SMA / Agentic AI	10
2.3.1	Éléments communs	10
2.3.2	Différences structurelles	11

2.3.3	Apports croisés	11
2.4	Synthèse et positionnement du projet	12
3	Choix du framework	13
3.1	Critères d'évaluation	13
3.2	Frameworks candidats	14
3.2.1	LangGraph	14
3.2.2	LangChain	14
3.2.3	AutoGen	15
3.2.4	CrewAI	15
3.2.5	LlamaIndex	15
3.2.6	MetaGPT	15
3.2.7	OpenAI Swarm	15
3.2.8	AWS Strands	15
3.2.9	Semantic Kernel	16
3.3	Comparaison synthétique	16
3.4	Décision motivée : LangGraph	16
3.5	Tradeoffs assumés	17
4	Cas d'étude : SMA de recrutement	18
4.1	Cahier des charges	18
4.1.1	Entrée	18
4.1.2	Sorties	18
4.1.3	Exigences fonctionnelles	18
4.1.4	Exigences non-fonctionnelles	19
4.2	Cas d'usage	19
4.3	Vue d'ensemble de l'architecture	20
4.3.1	Topologie en huit agents	20
4.3.2	Flux du pipeline	21
4.3.3	État partagé	23
4.3.4	Stack technique	25
4.4	Périmètre du système	26
4.4.1	Inclus dans le périmètre	26
4.4.2	Hors du périmètre	26

5	Analyse du système	27
5.1	Architecture du graphe LangGraph	27
5.2	Patterns de coordination retenus	28
5.3	Justification des choix d'implémentation	29
5.3.1	StateGraph et état typé plutôt que MessageGraph	29
5.3.2	Send/Reduce pour l'évaluation parallèle	30
5.3.3	Interrupt_before sur A7	30
5.3.4	Routage conditionnel à seuils absolu et relatif	30
5.3.5	Séparation A4 (Évaluateur) / A5 (Vérificateur)	31
5.3.6	Ollama local plutôt que API externe (Anthropic, OpenAI)	31
5.3.7	Découpage en huit agents spécialisés	32
5.3.8	Checkpointing (MemorySaver)	32
5.4	Extraits commentés	32
5.5	Pipeline de sourcing multi-sources (A3)	35
5.6	Filtre anti-bruit algorithmique	36
5.7	Mémoire RAG calibrée par fiche de poste	37
5.7.1	Architecture à deux collections	37
5.7.2	Procédure de lecture en deux temps	38
5.7.3	Garanties testées	38
5.8	Justification des choix techniques	39
5.8.1	Sources gratuites uniquement (DuckDuckGo, GitHub, Stack Overflow)	39
5.8.2	Appels d'outils déterministes plutôt que ToolNode/bind_tools	39
5.8.3	Filtrage algorithmique plutôt que LLM	40
5.8.4	Décomposition A3 en trois sous-agents	40
5.8.5	<i>Embeddings</i> locaux (SentenceTransformers all-MiniLM-L6-v2)	40
5.8.6	ChromaDB local plutôt que service externe	41
5.8.7	RAG calibré plutôt que RAG global naïf	41
5.9	Extraits commentés	41
5.10	Architecture API Gateway	44
5.11	Streaming temps réel via SSE	45
5.12	Human-in-the-loop : signalement par threading.Event	46
5.13	Interface graphique NiceGUI	47
5.14	Observabilité et tests	48
5.14.1	Collecte et export de métriques	48

5.14.2	Journalisation structurée	49
5.14.3	Couverture de tests	49
5.14.4	Conteneurisation et déploiement	49
5.15	Justification des choix techniques	50
5.16	Extraits commentés	50
5.17	Synthèse du chapitre	51
6	Conclusion et perspectives	53
6.1	Bilan	53
6.2	Limites identifiées	54
6.3	Perspectives	55
6.4	Considérations éthiques et RGPD	55
	Bibliographie	57

Chapitre 1

Présentation du projet

1.1 Contexte du travail

Ce rapport présente le Travail d'Étude et de Recherche (TER) de Master 1 réalisé au cours de l'année universitaire 2025–2026. Le TER constitue, dans le cursus, le premier exercice de recherche appliquée d'envergure : il prolonge des enseignements théoriques (intelligence artificielle, systèmes distribués, génie logiciel) par un travail combinant état de l'art, conception et réalisation, le tout justifié par une démarche argumentative.

Le sujet a été choisi à l'intersection de deux paradigmes vivants de l'intelligence artificielle. D'un côté, les *Systèmes Multi-Agents* (SMA) classiques, formalisés depuis les années 1990 et reposant sur des fondements mathématiques solides : autonomie, coordination par protocoles formels, apprentissage distribué [1, 2]. De l'autre, l'*Agentic AI*, paradigme émergent qui mobilise les grands modèles de langage (LLM) comme couche cognitive d'agents capables de planifier, d'utiliser des outils et de mémoriser un contexte persistant [3, 4].

Plutôt que d'opposer ces deux approches, le présent travail cherche à les articuler dans un cas d'étude concret. La littérature analysée (voir chapitre 2) montre en effet une convergence : les SMA fournissent des protocoles formels et des garanties de stabilité ; l'Agentic AI apporte une flexibilité cognitive issue des LLM. Le cas d'étude retenu, un système d'aide au recrutement, sollicite naturellement ces deux dimensions.

1.2 Problématique fonctionnelle

Le sourcing et l'évaluation initiale de candidats sont des activités encore largement manuelles dans les processus de recrutement informatique. Pour une fiche de poste donnée, un recruteur doit (i) identifier les sources pertinentes, (ii) collecter des profils, (iii) éliminer le bruit (offres d'emploi, profils non pertinents), (iv) évaluer chaque profil au regard du besoin, (v) prioriser une liste de candidats à contacter. Ce cycle est lent, sujet à biais cognitifs et difficilement traçable lorsqu'il est répété sur plusieurs postes.

L'automatisation totale de ce processus poserait toutefois des problèmes éthiques et légaux

importants : décision algorithmique opaque, transmission de données personnelles à des services tiers (RGPD), risque de biais discriminatoires sans contrôle humain. La problématique de ce TER est donc plus précise :

Comment concevoir un système d'aide au recrutement qui automatise les tâches mécaniques (collecte, déduplication, première évaluation), tout en conservant explicitement la souveraineté décisionnelle humaine, en garantissant l'auditabilité de chaque étape et en respectant la confidentialité des données ?

Cette formulation conditionne directement plusieurs choix d'architecture explicités dans le rapport : un point d'arrêt obligatoire avant tout contact effectif (*human-in-the-loop*), une exécution locale du modèle de langage pour éviter l'export de données candidats, une décomposition en agents spécialisés permettant la traçabilité par étape.

1.3 Objectifs du TER

Le travail poursuit trois objectifs articulés.

Objectif théorique. Conduire un état de l'art structuré sur les SMA classiques et l'Agentic AI, et produire une analyse comparative argumentée des deux paradigmes. Cet objectif est traité dans le chapitre 2, à partir d'un corpus d'études rassemblées dans le dossier *Etudes/* du projet (six études SMA, onze études Agentic AI, et une synthèse comparative).

Objectif méthodologique. Sélectionner un *framework* d'orchestration d'agents adapté au cas d'étude, en justifiant chaque critère d'évaluation et chaque comparaison. Le chapitre 3 détaille cette démarche, qui s'appuie sur un comparatif détaillé des frameworks d'IA agentic.

Objectif applicatif. Concevoir, implémenter et évaluer un système multi-agents complet pour le recrutement. Le système, présenté au chapitre 4 et analysé en profondeur au chapitre 5, comporte huit agents spécialisés, une mémoire vectorielle calibrée, une interface graphique avec suivi temps réel, et une suite de tests automatisés. Tous les choix techniques sont assortis d'une justification *ex post*.

Une contrainte transversale, énoncée par l'équipe et le commanditaire pédagogique, structure l'ensemble du document : **chaque choix doit être justifié explicitement**, en mentionnant le besoin auquel il répond, les alternatives considérées, la décision retenue et les compromis assumés.

1.4 Démarche et plan du rapport

Le rapport est organisé en six chapitres.

Chapitre 1.

Présente le contexte, la problématique et les objectifs.

Chapitre 2.

Conduit l'état de l'art SMA / Agentic AI et l'analyse comparative.

Chapitre 3.

Justifie le choix du *framework* LangGraph face aux alternatives.

Chapitre 4.

Spécifie le cas d'étude et donne une vue d'ensemble du système.

Chapitre 5.

Analyse en détail l'implémentation, regroupée en trois pans (cœur LangGraph, outils et RAG, couche service et opérations).

Chapitre 6.

Dresse le bilan, les limites et les perspectives.

L'équipe est composée de trois étudiants. Le périmètre de chacun est rappelé dans la conclusion. Le code source, l'historique Git, les tests automatisés et les exports d'observabilité accompagnent ce rapport et peuvent être consultés pour vérification.

Le travail s'est étalé sur seize semaines, organisées en cinq phases qui se chevauchent volontairement (figure 1.1) : recherche bibliographique, conception, implémentation, validation et restitution.

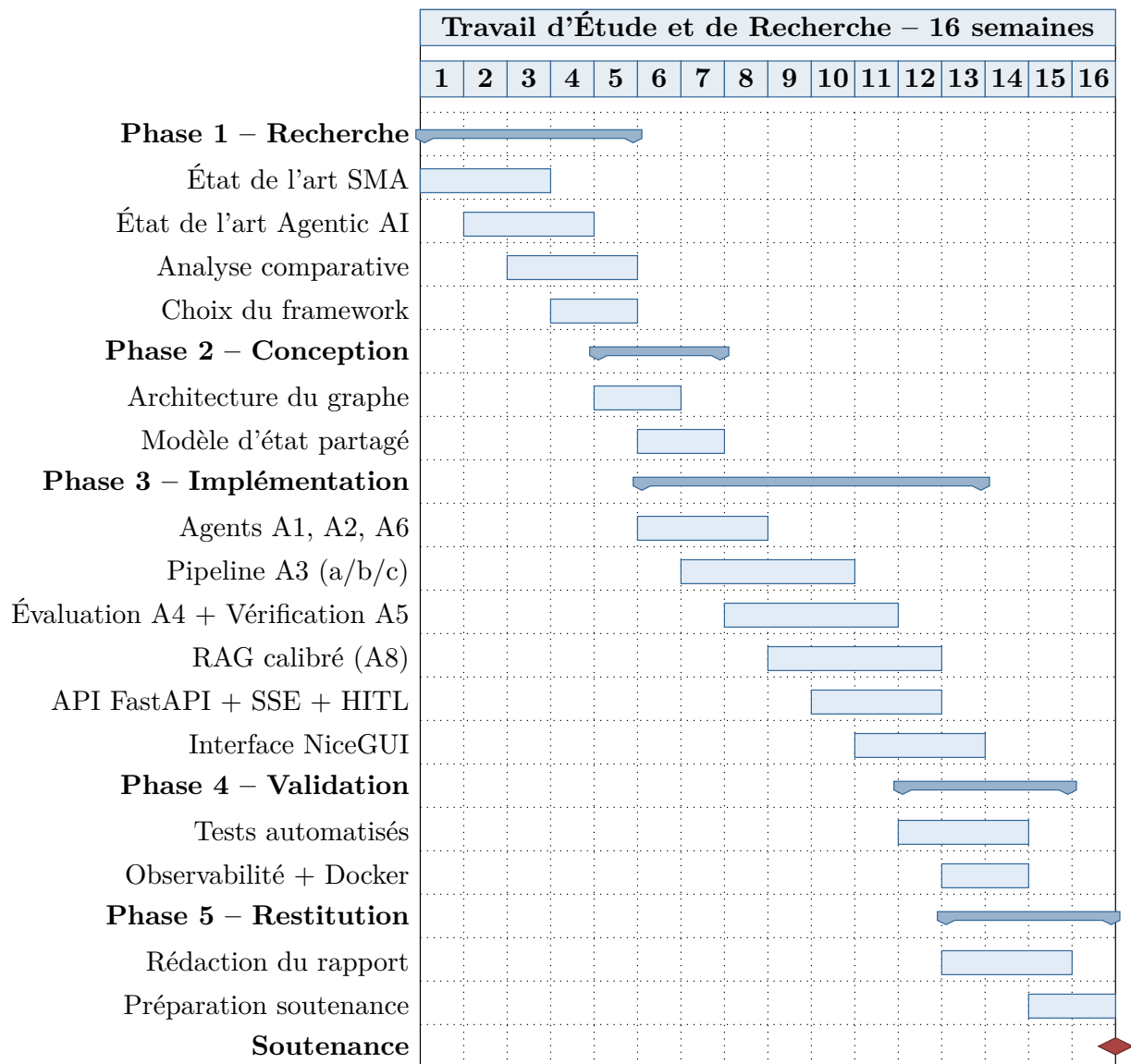


FIGURE 1.1 – Planning prévisionnel du TER découpé en cinq phases (recherche, conception, implémentation, validation, restitution). Les phases se chevauchent volontairement : l'implémentation commence dès que la conception est suffisamment stable, la rédaction du rapport démarre pendant la phase d'implémentation pour absorber les retours de relecture.

Chapitre 2

État de l’art

Ce chapitre rend compte du travail bibliographique conduit pendant les premières semaines du TER. Il s’appuie sur dix-huit études rassemblées dans le dossier **Etudes/** du projet (six études dédiées aux SMA, onze études dédiées à l’Agentic AI, plus une synthèse comparative) et résume les invariants, les divergences et les ponts entre les deux paradigmes. La section 2.1 traite des SMA classiques, la section 2.2 de l’Agentic AI, la section 2.3 de l’analyse comparative, et la section 2.4 positionne le projet à l’intersection des deux courants.

2.1 Systèmes Multi-Agents (SMA) classiques

2.1.1 Définition et composants fondamentaux

Selon DORRI, KANHERE et JURDAK [1], un *Système Multi-Agents* est un ensemble d’agents autonomes interagissant dans un environnement partagé en vue d’objectifs individuels ou collectifs. MALDONADO et al. [5] complètent en identifiant cinq composants structurants :

- **Agents** : entités autonomes capables de percevoir et d’agir ;
- **Environnement** : espace partagé dans lequel les agents évoluent ;
- **Interactions** : échanges et communications entre agents ;
- **Organisation** : structure régissant les rôles et les relations ;
- **Objectifs** : buts individuels ou collectifs poursuivis.

2.1.2 Propriétés des agents

Un agent au sein d’un SMA est conventionnellement caractérisé par quatre propriétés [5, 2] :

Autonomie

décision indépendante sans intervention extérieure ;

Réactivité

capacité à répondre aux variations de l’environnement ;

Proactivité

poursuite active d’objectifs définis ;

Sociabilité

aptitude à communiquer et à coopérer.

Ces quatre propriétés sont des points d’évaluation directement transposables à l’Agentic AI : un système agentique qui les satisfait simultanément peut prétendre à un statut d’agent au sens classique du terme.

2.1.3 Architectures organisationnelles et cognitives

Les SMA peuvent adopter différentes architectures organisationnelles selon les contraintes du domaine [1] :

Centralisée

un agent maître coordonne les agents subordonnés ;

Décentralisée

la coordination émerge des interactions locales ;

Hiérarchique

organisation multi-niveaux combinant coordination locale et supervision globale.

MALDONADO et al. [5] distinguent en parallèle trois types d’architectures cognitives :

Réactifs

agents simples répondant directement aux stimuli ;

Cognitifs

agents dotés d’une représentation interne et d’une capacité de planification ;

Hybrides

combinaison des deux approches.

Le système de recrutement étudié dans ce rapport relève sans ambiguïté de la catégorie des architectures hiérarchiques cognitives : un orchestrateur (A1) supervise des agents spécialisés, chacun doté d’une représentation explicite de l’état partagé (**GraphState**) et d’une capacité de planification (par *prompt* structuré).

2.1.4 Coordination, négociation et protocoles formels

Les travaux du *General Survey* [2] formalisent les interactions multi-agents par plusieurs mécanismes éprouvés :

- **Contract Net Protocol** : appel d’offres pour l’allocation de tâches ;
- **Théorie des jeux** : modélisation des stratégies et des équilibres ;
- **Mécanismes d’enchères** : allocation optimale de ressources ;
- **Consensus distribué** : convergence vers un accord collectif.

DORRI, KANHERE et JURDAK [1] soulignent par ailleurs le rôle structurant de l’*Agent Communication Language* (ACL) standardisé par la FIPA [6] : performatives, ontologie partagée, intentions explicites. Ces protocoles garantissent trois propriétés critiques pour les systèmes distribués : stabilité, optimalité, convergence.

Le présent projet n’implémente pas de protocole FIPA-ACL stricto sensu. Il en hérite cependant l’esprit : la communication entre agents passe par un schéma typé (`TypedDict`), chaque champ porte une sémantique métier explicite, et les écritures concurrentes sont synchronisées par des *reducers* formellement définis (cf. chapitre 5).

2.1.5 Apprentissage : du SMA au MARL

Le *CMU Paper* [7] établit un pont entre les SMA classiques et le *Multi-Agent Reinforcement Learning* (MARL). Quatre défis principaux y sont identifiés : non-stationnarité (l’environnement évolue selon les actions de tous les agents), instabilité de convergence (mise à jour conjointe des politiques), équilibre coopération / compétition, et exploration collective sans duplication.

Notre projet n’introduit pas d’apprentissage en ligne entre agents : le LLM est utilisé en mode inférence seule, sans *fine-tuning*. Le RAG calibré par fiche (cf. chapitre 5) constitue toutefois un mécanisme rudimentaire d’amélioration inter-runs : les évaluations passées, conditionnées au contexte de leur fiche, alimentent un signal de calibration pour les évaluations suivantes.

2.1.6 Émergence et intelligence collective

Le *Visual Survey* [8] met en lumière la propriété d’émergence : des comportements globaux complexes naissent d’interactions locales simples (trafic routier, marchés financiers, propagation d’épidémies). L’intelligence collective dépasse alors la somme des capacités individuelles.

Cette propriété est moins directement applicable au cas du recrutement, qui ne mobilise pas un grand nombre d’agents homogènes. On pourrait néanmoins la rapprocher, à un niveau plus modeste, du pattern *Send/Reduce* mis en œuvre dans le système : N instances de l’évaluateur travaillent en parallèle puis convergent par un agrégateur.

2.1.7 Applications

Les SMA trouvent des applications dans des domaines variés [9] : *smart grids*, véhicules autonomes, robotique collective, blockchain, simulation sociale, *smart cities*, Industrie 4.0. Le recrutement automatisé n’apparaît pas dans cette liste canonique : il est plus naturellement classé du côté de l’IA générative et des agents conversationnels. Le présent travail, en mêlant les deux courants, contribue à élargir le spectre applicatif.

2.2 Agentic AI : définition et fondements

2.2.1 De l’agent à l’agenticité

Plusieurs travaux récents convergent vers une clarification conceptuelle : l’Agentic AI n’est ni un simple agent LLM augmenté d’outils, ni une réincarnation des SMA classiques [3, 10, 11]. Trois niveaux émergent :

AI Agents [3]

systèmes modulaires activés par des LLM, spécialisés dans l’exécution de tâches définies via intégration d’outils et *prompt engineering* ;

Agentic AI [10, 12]

paradigme orienté objectifs, caractérisé par la planification autonome, la mémoire persistante et l’adaptation dynamique ;

Communautés agentiques [11]

écosystèmes hybrides où agents IA et humains interagissent via des rôles formels et des mécanismes de gouvernance.

L’agenticité correspond moins à une technologie spécifique qu’à un changement de philosophie computationnelle : le passage d’une génération réactive à une action intentionnelle structurée.

2.2.2 Double paradigme : symbolique et neuronal

ALI et al. [4] proposent un cadre interprétatif à double paradigme :

Paradigme symbolique

planification déterministe, état persistant, vérification formelle, forte prédictibilité (souvent dominant en santé critique) ;

Paradigme neuronal

orchestration par LLM, génération probabiliste, adaptabilité élevée (prévalent en finance et environnements dynamiques).

Les travaux convergent vers une trajectoire neuro-symbolique hybride combinant adaptabilité (LLM), robustesse logique (symbolique) et vérifiabilité (méthodes formelles). Le projet présenté ici relève d’une telle hybridation : la topologie du graphe et les contrats d’état sont symboliques (déterministes, typés, testables), tandis que les nœuds A2, A3a, A4 et A5 reposent sur un LLM probabiliste.

2.2.3 Architecture générale d’un système agentique

Les études techniques [13, 14, 15] décrivent une architecture modulaire récurrente composée de : un moteur de raisonnement (LLM comme cœur cognitif), une mémoire persistante (court et long terme), une planification multi-étapes, une sélection dynamique d’outils, et une boucle itérative analyse-plan-action-évaluation. La figure 2.1 en donne une représentation synthétique.

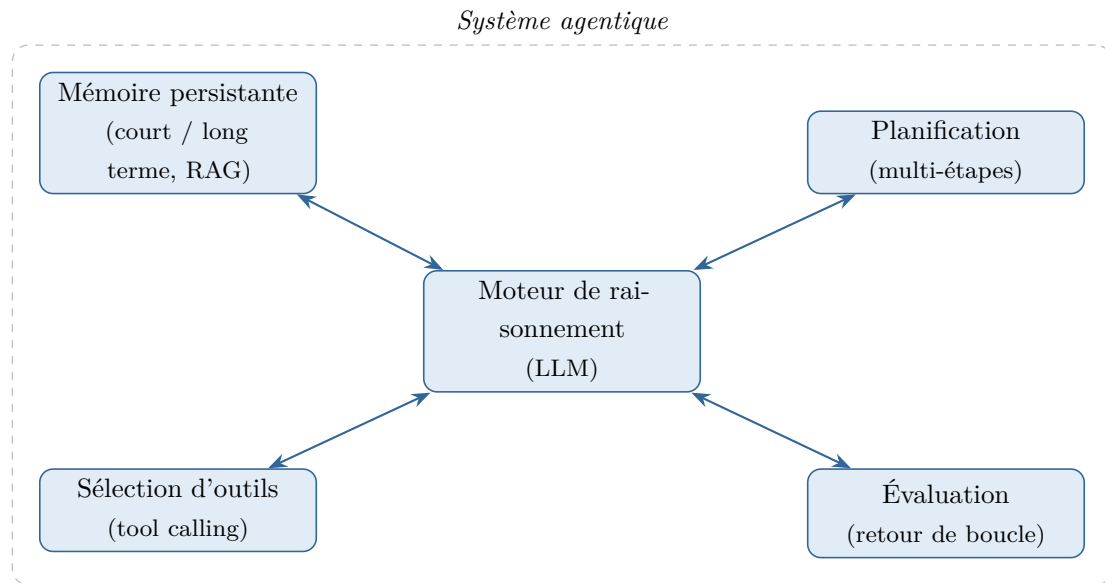


FIGURE 2.1 – Architecture modulaire récurrente d’un système agentique : un LLM cognitif au centre, entouré d’une mémoire persistante, d’un planificateur, d’un sélecteur d’outils et d’un module d’évaluation refermant la boucle [13, 14, 15].

L’industrialisation de tels systèmes requiert des registres de modèles et d’agents (*Agent Factory*), une orchestration *cloud-native*, des microservices distribués, et des mécanismes de *guardrails*. VAIDHYANATHAN et TAIBI [15] relèvent toutefois plusieurs faiblesses persistantes : modèles de mémoire limités, planification rudimentaire, écosystème fragmenté.

2.2.4 Du single-agent aux systèmes multi-agents agentiques

MILOSEVIC et RABHI [11] et HUGHES et al. [16] montrent que l’évolution naturelle de l’Agentic AI est la coordination multi-agents, sous trois formes :

- orchestration hiérarchique (manager → experts) ;
- collaboration horizontale ;
- intégration humain-dans-la-boucle.

Contrairement aux SMA classiques, ces systèmes utilisent des LLM comme couche cognitive, intègrent une décomposition dynamique des tâches et introduisent une autonomie adaptative plutôt qu’un échange protocolaire. Le système de recrutement implémenté combine ces trois formes : orchestration hiérarchique (A1 supervise), collaboration horizontale (A4 et A5 coopèrent en validation pair-à-pair), et HITL natif via `interrupt_before`.

2.2.5 Applications structurantes

Les domaines récurrents identifiés sont : santé (aide au diagnostic, déterminisme requis), finance (traitement dynamique de flux massifs), robotique (perception-planification hybride), génie logiciel (agents de codage [17]), automatisation d’entreprise (orchestration décisionnelle).

2.2.6 Enjeux transversaux

Plusieurs défis structurants émergent de manière convergente [18, 12, 15] :

- hallucination et fragilité des LLM ;
- sécurité (*prompt injection*, fuite de données) ;
- responsabilité et attribution des décisions ;
- consommation énergétique ;
- alignement éthique et gouvernance réglementaire.

La montée en autonomie accroît mécaniquement la nécessité de vérification formelle, de supervision humaine et de cadres organisationnels robustes.

2.2.7 Trajectoire évolutive

La convergence des articles suggère une trajectoire claire [10, 11], illustrée figure 2.2 :

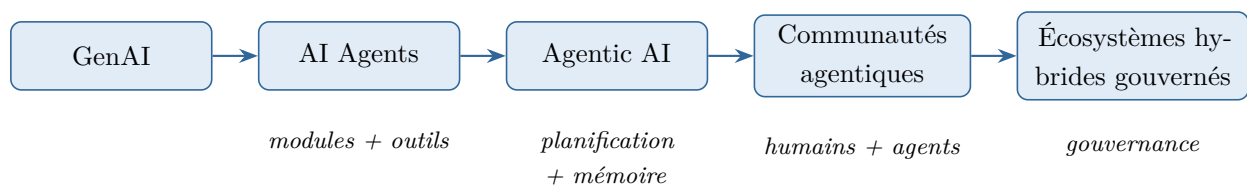


FIGURE 2.2 – Trajectoire évolutive identifiée par la littérature : de l’IA générative aux écosystèmes hybrides gouvernés [10, 11].

L’Agentic AI n’est pas une simple extension des LLM : c’est une transformation systémique de l’architecture décisionnelle, marquant le passage d’une intelligence générative à une intelligence organisationnelle distribuée.

2.3 Analyse comparative SMA / Agentic AI

L’analyse comparative repose sur le document `analyse_comparative_SMA_AgenticAI.pdf` produit dans le dossier `Etudes/`. Trois sections la structurent : éléments communs, différences structurelles, apports croisés.

2.3.1 Éléments communs

Cinq invariants relient les deux paradigmes :

1. **Autonomie des agents** : décisions sans intervention extérieure [1, 3].
2. **Poursuite active d’objectifs** : action proactive et non simplement réactive [5, 12].
3. **Collaboration multi-agents** : résolution de problèmes complexes par coordination [5, 11].

4. **Mémoire et persistance du contexte** : cohérence des décisions dans le temps [2, 14].
5. **Domaines d’application communs** : santé, robotique, véhicules autonomes, automatisation d’entreprise [9, 16, 4].

2.3.2 Différences structurelles

Cinq différences les opposent :

Dimension	SMA classiques	Agentic AI
Nature du raisonnement	Logique, symbolique, déterministe, formellement prouvable	Probabiliste (LLM), flexible, non garanti
Mode de coordination	Protocoles formels (Contract Net, enchères, ACL)	Orchestration souple (<i>prompt engineering, tool calling</i>)
Rôle de l’apprentissage	Optionnel, extension via MARL	Central : LLM = produit d’un entraînement massif
Maturité	Décennies de recherche, fondements mathématiques solides	Écosystème fragmenté, mémoire limitée, interopérabilité inégale
Explicabilité	Inhérente : règles et protocoles traçables	Déficit structurel des LLM, défi majeur de gouvernance

TABLE 2.1 – Différences structurelles entre SMA classiques et Agentic AI [4, 2, 10, 15].

2.3.3 Apports croisés

L’analyse comparative dégage cinq apports possibles des SMA à l’Agentic AI :

1. **Protocoles formels de coordination** : Contract Net, enchères et consensus pour structurer la collaboration entre agents LLM [2, 7, 15].
2. **Stabilisation par la théorie des jeux** : modéliser les interactions d’agents LLM, anticiper les conflits, garantir des équilibres stables [2, 10].
3. **Formalisation par le modèle BDI** : Beliefs $\rightarrow$ mémoire contextuelle, Desires $\rightarrow$ objectifs, Intentions $\rightarrow$ plan d’action [5, 3, 10].
4. **Gouvernance et vérification formelle** : combler le déficit de gouvernance des systèmes LLM [11, 16, 18].
5. **Apprentissage coordonné via MARL** : améliorer la coordination dans des environnements dynamiques [7, 3].

Les SMA ne s’opposent pas à l’Agentic AI : ils constituent un réservoir de solutions formelles et éprouvées capable de combler ses faiblesses actuelles en matière de stabilité, de gouvernance et d’explicabilité (synthèse Etudes/analyse_comparative_SMA_AgenticAI.pdf).

2.4 Synthèse et positionnement du projet

L’état de l’art conduit dans ce chapitre permet de positionner clairement le projet à l’intersection des deux paradigmes.

Héritage SMA classique. Le système hérite : (i) d’une organisation hiérarchique cognitive (orchestrateur supervisant des agents spécialisés) ; (ii) d’une coordination par état partagé typé, héritière du *blackboard* ; (iii) d’un patron de validation pair-à-pair (A4/A5) qui rappelle les mécanismes de *checks and balances* formels ; (iv) d’une exécution déterministe et tracée, propice à l’audit.

Mobilisation de l’Agentic AI. Le système emploie : (i) un LLM comme couche cognitive de plusieurs agents (analyse, recherche, évaluation, vérification, rédaction) ; (ii) une boucle planifier-agir-évaluer locale à chaque agent ; (iii) une mémoire vectorielle persistante (RAG) ; (iv) un *tool calling* structuré (recherche web, scraping, base ChromaDB) ; (v) un point d’arrêt humain explicite avant toute action irréversible.

Contribution. Le présent travail apporte une démonstration concrète qu’un cas d’étude métier (recrutement) peut être traité en exploitant simultanément la rigueur formelle des SMA (typage d’état, *reducers*, routage conditionnel, HITL) et la flexibilité cognitive de l’Agentic AI (raisonnement par LLM, RAG, *tool use*). Cette articulation est argumentée tout au long du rapport, et chaque choix d’implémentation est rapporté à l’un ou l’autre des deux paradigmes.

Chapitre 3

Choix du framework

Ce chapitre justifie le choix du *framework* d'orchestration d'agents retenu pour l'implémentation. La démarche s'appuie sur le tableau comparatif détaillé conservé dans le dossier `Etudes/`(`comparaison_frameworks_IA_detaill  .xlsx`). La section 3.1 formalise les critères d'évaluation, la section 3.2 présente les candidats et leurs caractéristiques, la section 3.4 formule la décision motivée, et la section 3.5 explicite les compromis assumés.

3.1 Critères d'évaluation

Les critères ont été construits à partir des concepts opératoires identifiés dans le glossaire technique du dossier d'études, en les hiérarchisant selon le besoin du cas d'étude.

Architecture en graphe orienté. Un graphe orienté est la structure la plus expressive pour modéliser un SMA : il permet d'identifier précisément qui communique avec qui, dans quel ordre, sous quelles conditions ; il est traçable, débogable, et proche de la théorie formelle des SMA. C'est un critère décisif pour un cas d'étude où l'auditabilité est une exigence.

État persistant typé (*graph state*). Sans état persistant, un SMA oublie tout entre exécutions. Avec un état persistant, le système conserve une mémoire cohérente, peut être interrompu et repris (HITL, crash) et peut être audité *post mortem*. La persistance doit être typée pour garantir l'intégrité des contrats inter-agents.

Paradigme de coordination explicite. Une coordination mal définie produit des agents redondants, contradictoires ou bloqués. Quatre paradigmes sont possibles : centralisé, hiérarchique, pair-à-pair, ou *LLM-first* (le modèle décide en temps réel). Pour un cas d'étude critique, un modèle explicite et auditable est préférable à un modèle entièrement *LLM-first*.

Human-in-the-loop natif. Mécanisme permettant à un humain d'intervenir en cours d'exécution. Crucial pour des applications où une mauvaise décision a des conséquences réelles,

comme le contact d'un candidat. Le *framework* doit supporter ce mécanisme nativement, sans contournement.

Intégration RAG et bases vectorielles. Le cas d'étude exige une mémoire vectorielle (calibration par fiche). Le *framework* retenu doit s'interfacer simplement avec une base de type Chroma, Qdrant ou Pinecone.

Indépendance vis-à-vis du fournisseur. Le *vendor lock-in* (dépendance à OpenAI, AWS, Microsoft) est un risque stratégique. Si le fournisseur augmente ses prix, arrête un service ou change ses conditions, l'infrastructure est à refaire. Pour un projet académique souverain, l'indépendance est un critère fort.

Licence et coût d'exploitation. Une licence permissive (MIT ou Apache 2.0) permet une utilisation libre. Le coût d'exploitation effectif dépend également des services associés : un *framework* libre couplé à une plateforme propriétaire reproduit le *lock-in*.

Courbe d'apprentissage. Pour un TER de quelques mois, une courbe d'apprentissage maîtrisable est nécessaire. Cependant, ce critère ne doit pas l'emporter sur la puissance d'expression : un *framework* trop simple amputerait l'argumentation architecturale.

Maturité de l'écosystème. Documentation, communauté, intégrations, exemples, retours de production. Un *framework* en *alpha* est risqué pour un projet à délai contraint.

3.2 Frameworks candidats

Neuf *frameworks* sont confrontés aux critères. Les caractéristiques rapportées s'appuient sur le tableau comparatif détaillé du dossier d'études [15].

3.2.1 LangGraph

Bibliothèque de la suite LangChain spécialisée dans la composition d'agents [19]. Caractéristiques : architecture en graphe orienté de premier ordre (nœuds, arêtes, branches conditionnelles, boucles) ; état persistant typé via `TypedDict` ; *checkpointing* natif (en mémoire ou sur SQLite) ; mécanisme `interrupt_before` dédié au HITL ; intégrations natives avec ChromaDB et autres bases vectorielles ; modèle d'orchestration explicite, indépendant du fournisseur LLM. Licence MIT. Courbe d'apprentissage moyenne à élevée.

3.2.2 LangChain

Couche de plus haut niveau [20] pour construire des chaînes et agents. Très large adoption, écosystème riche. La structure d'une chaîne reste cependant linéaire ou faiblement branchante ;

LangChain n'expose pas les mêmes primitives de graphe ni le même contrôle d'état que LangGraph. Souvent utilisée comme socle, complétée par LangGraph pour des topologies non triviales.

3.2.3 AutoGen

Framework Microsoft Research [21] centré sur la conversation multi-agents : les agents échangent des messages, un *group chat* orchestre les tours. Supporte le HITL nativement. Très expressif pour des dialogues d'agents, mais le modèle conversationnel ne se prête pas naturellement à un pipeline en pas successifs avec étapes critiques (déduplication, scoring, routage par seuil).

3.2.4 CrewAI

Framework récent [22] centré sur la composition de rôles (*crew*) et de tâches. S'appuie sur LiteLLM, donc indépendant du fournisseur. Faible courbe d'apprentissage. La coordination repose sur une orchestration souple (*LLM-first*) qui rend l'audit pas-à-pas plus difficile que dans un modèle de graphe explicite.

3.2.5 LlamaIndex

Framework [23] excellent en RAG et indexation de corpus documentaires, avec une couche d'agents plus modeste. C'est davantage un moteur de recherche intelligent qu'un orchestrateur multi-agents.

3.2.6 MetaGPT

Framework structuré autour de *Standard Operating Procedures* (SOP) : enchaînement d'étapes prédéfinies imitant un cycle de production logicielle. Garantit la reproductibilité dans des environnements stables, mais la rigidité des SOP est mal adaptée à un pipeline de recrutement où le routage dépend dynamiquement des scores produits.

3.2.7 OpenAI Swarm

Bibliothèque très légère orientée *handoffs* (passages de relais) entre agents. Simplicité didactique, mais : pas d'état partagé riche, pas de retour en arrière, dépendance forte à OpenAI (*vendor lock-in*). Inadéquat pour le cahier des charges.

3.2.8 AWS Strands

Framework centré sur AWS Bedrock, exploitant le protocole MCP. Adoption hors AWS limitée. Crée un *vendor lock-in* sur AWS qui contredit l'objectif d'indépendance et de souveraineté.

3.2.9 Semantic Kernel

Framework Microsoft fortement intégré à Azure et au stack .NET. Excellent dans son écosystème, mais reproduit la dépendance fournisseur sur Azure. Inadéquat pour un déploiement local et indépendant.

3.3 Comparaison synthétique

Framework	Graphe	État typé	HITL natif	Indépendance	Maturité	Adéquation au cas d'étude
LangGraph	oui	oui	oui	oui	élevée	adéquat (architecture en graphe + HITL + RAG)
LangChain	non	limité	partiel	oui	élevée	insuffisant pour topologie complexe
AutoGen	non	non	oui	oui	moyenne	inadéquat : modèle conversationnel
CrewAI	non	limité	partiel	oui	moyenne	inadéquat : audit pas-à-pas faible
LlamaIndex	non	non	non	oui	élevée	complémentaire (RAG) mais pas orchestrateur
MetaGPT	SOP	non	non	oui	moyenne	inadéquat : SOP trop rigides
OpenAI Swarm	non	non	non	non	faible	inadéquat : <i>lock-in</i> OpenAI
AWS Strands	oui	oui	oui	non	moyenne	inadéquat : <i>lock-in</i> AWS
Semantic Kernel	non	oui	oui	non	élevée	inadéquat : <i>lock-in</i> Azure

TABLE 3.1 – Comparatif synthétique des *frameworks* confrontés aux critères du cas d'étude.

3.4 Décision motivée : LangGraph

Le choix de LangGraph résulte de la conjonction de cinq éléments.

Adéquation structurelle. LangGraph est le seul *framework* parmi les indépendants à offrir simultanément : une architecture en graphe orienté de premier ordre, un état persistant typé, un mécanisme HITL natif, une intégration directe à ChromaDB, et un *checkpoint* permettant la reprise d'exécution. Tous les autres candidats indépendants (LangChain, CrewAI, AutoGen) excellent sur un sous-ensemble de critères mais n'offrent pas la combinaison complète.

Convergence avec l'argumentation théorique. Le chapitre 2 a montré que les SMA classiques apportent à l'Agentic AI des protocoles formels, des garanties de stabilité et une vérification explicite. Une architecture en graphe orienté est précisément le support naturel de ces apports : nœuds et arêtes formalisent les contrats, le typage protège les invariants, le *checkpoint* permet l'audit. Le projet incarne ainsi l'apport SMA → Agentic AI dans son choix de *framework*.

Indépendance et souveraineté. LangGraph n’est lié à aucun fournisseur LLM particulier : il s’interface aussi bien avec Ollama (local) qu’avec OpenAI, Anthropic ou tout fournisseur compatible LangChain. Cette indépendance est cohérente avec le choix d’Ollama local pour la confidentialité des données candidats.

Maturité et licence. Bibliothèque MIT, communauté active, documentation officielle riche, exemples nombreux. Ces éléments réduisent le risque temporel pour un TER à délai contraint.

Expressivité argumentative. Pour un jury évaluant l’argumentation architecturale, LangGraph fournit un vocabulaire précis et démontrable : `StateGraph`, `Send/Reduce`, `interrupt_before`, `conditional edges`, `checkpoint`. Chacun de ces concepts cristallise un patron de coordination qui peut être exhibé et défendu.

3.5 Tradeoffs assumés

Trois compromis sont explicitement assumés par ce choix.

Courbe d’apprentissage. LangGraph est plus exigeant qu’un *framework LLM-first* comme CrewAI. Il faut comprendre les notions de *StateGraph*, de *reducer*, de *checkpoint*, de *conditional edges*. Cet investissement initial est compensé par la lisibilité du code obtenu : un nouveau lecteur peut reconstruire mentalement le pipeline en lisant le fichier `src/graph.py` (environ deux cents lignes).

Verbosité du typage d’état. Le `TypedDict GraphState` comporte une quinzaine de champs explicites. Ce typage explicite est plus verbeux qu’un état implicite à la AutoGen, mais chaque champ documente précisément le contrat entre agents et permet une détection statique des fautes (champ inexistant, type incompatible).

Dépendance à l’écosystème LangChain. LangGraph s’inscrit dans la suite LangChain, dont l’évolution rapide implique des ruptures occasionnelles d’API. Le projet épingle des versions précises (`langgraph>=1.1.0`, `langchain>=0.3.0`) et délimite l’usage de LangChain au strict nécessaire (initialisation du modèle via `init_chat_model`). Le couplage est suffisamment limité pour qu’une migration ultérieure soit envisageable.

En conclusion, LangGraph satisfait simultanément les exigences d’expressivité (graphe, état, routage), d’auditabilité (typage, *checkpoint*), de gouvernance (HITL natif), d’indépendance (pas de *vendor lock-in*) et de cohérence avec l’argumentation théorique du chapitre précédent. Le chapitre suivant détaille comment ce choix se concrétise dans le cas d’étude.

Chapitre 4

Cas d'étude : SMA de recrutement

Ce chapitre spécifie le cas d'étude implémenté et donne une vue d'ensemble du système avant l'analyse détaillée du chapitre 5. La section 4.1 fixe le cahier des charges, la section 4.2 décrit les cas d'usage, la section 4.3 présente l'architecture des huit agents, et la section 4.4 précise le périmètre fonctionnel et non-fonctionnel.

4.1 Cahier des charges

4.1.1 Entrée

L'utilisateur (un recruteur) fournit en entrée une *fiche de poste* sous forme de texte libre : intitulé, missions, compétences attendues, niveau de séniorité, contexte (lieu, type de contrat, télétravail). Aucun format imposé : le texte peut être copié-collé d'une annonce ou rédigé pour la circonstance.

4.1.2 Sorties

Trois artefacts sont produits :

1. Une **liste de candidats classés** : chaque candidat est identifié par un nom, une ou plusieurs sources (URL d'origine), un score numérique entre 0 et 100, des explications structurées par axes (compétences, séniorité, contexte) et un statut de validation (*validé*, *douteux*, *invalide*).
2. Un **rapport synthétique** en Markdown : récapitule la fiche, les candidats retenus, les raisons du retenu/rejet, les statistiques d'exécution.
3. Le cas échéant, un ensemble de **messages de contact** pré-rédigés (un par candidat retenu), conditionnés à une validation humaine explicite.

4.1.3 Exigences fonctionnelles

EF-1 Sourcer des profils sur des sources publiques sans clé API payante.

EF-2 Filtrer le bruit (offres d'emploi, profils non pertinents) sans perdre de profils valides.

- EF-3** Évaluer chaque profil par rapport à la fiche, avec score et explications.
- EF-4** Effectuer un contrôle pair-à-pair : un agent vérificateur peut invalider ou ajuster un score.
- EF-5** Bloquer toute action irréversible (envoi de message) sans validation humaine explicite.
- EF-6** Mémoriser les évaluations passées pour calibrer les évaluations futures, sans introduire de biais inter-fiches.
- EF-7** Permettre un suivi en temps réel : avancement du pipeline, événements, métriques.

4.1.4 Exigences non-fonctionnelles

ENF-1

Confidentialité : les données candidats ne sortent pas du serveur ; le LLM est exécuté localement.

ENF-2

Auditabilité : chaque agent journalise ses entrées, ses sorties et ses métriques ; les exécutions sont *checkpointées*.

ENF-3

Reproductibilité : le système est livré avec un `docker-compose.yml` permettant un déploiement complet en une commande.

ENF-4

Robustesse : la suite de tests automatisés couvre les composants critiques (graphe, RAG, API, filtre, déduplicateur).

ENF-5

Démonstrabilité : le système doit pouvoir être démontré hors ligne devant un jury.

4.2 Cas d’usage

Trois cas d’usage principaux structurent l’interaction.

UC-1 — Lancer un sourcing. Le recruteur saisit une fiche de poste dans la page d’accueil de l’interface graphique, choisit s’il souhaite activer la validation humaine intermédiaire (HITL), puis lance le pipeline. Il est redirigé vers une page de suivi temps réel.

UC-2 — Suivre, valider et reprendre. Sur la page de suivi, le recruteur observe le diagramme du pipeline qui s’illumine au fur et à mesure de l’avancement. Lorsque le pipeline atteint le point d’arrêt HITL, l’interface affiche les candidats présélectionnés et propose trois actions : approuver la liste telle quelle, tout rejeter, ou éditer la liste (JSON éditable). L’action choisie reprend le pipeline exactement là où il s’était arrêté.

UC-3 — Consulter le rapport et la mémoire. Une fois le pipeline terminé, le recruteur peut consulter le rapport final, l'historique des *runs* précédents, et explorer la mémoire vectorielle (recherche de profils similaires, statistiques par fiche).

La figure 4.1 formalise l'ensemble des cas d'utilisation et des acteurs (le recruteur humain et les acteurs externes mobilisés par le système).

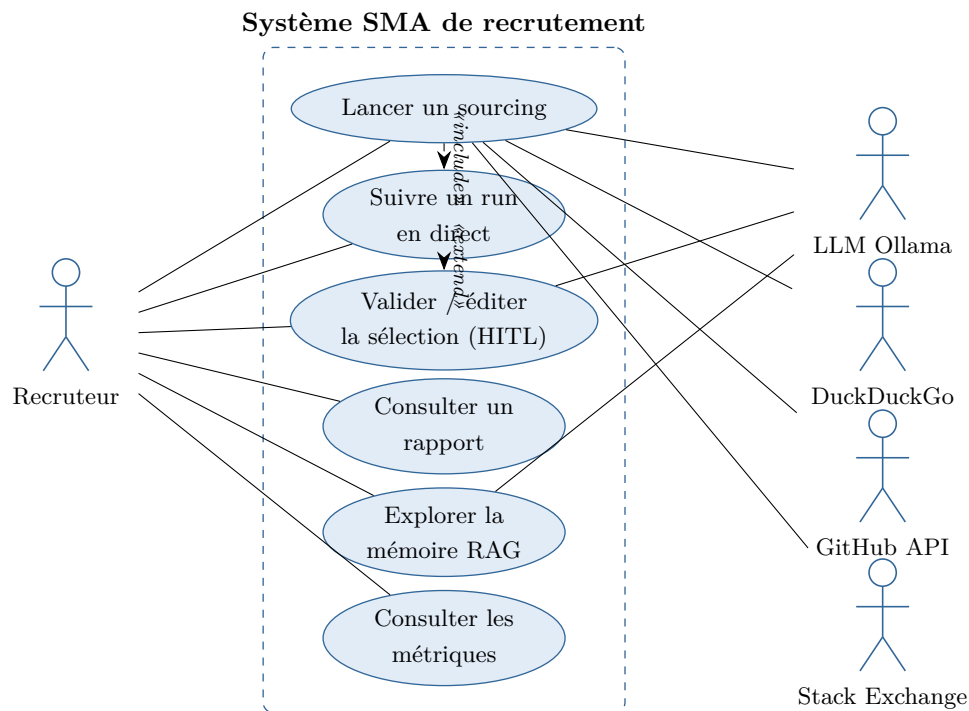


FIGURE 4.1 – Diagramme de cas d'utilisation. Le recruteur humain interagit avec six cas d'usage exposés par l'interface graphique. Le système mobilise quatre acteurs externes : le LLM Ollama (local) et trois sources publiques de *sourcing*. La relation *include* indique que tout lancement de *run* ouvre la page de suivi ; *extend* signale que la validation HITL n'apparaît que sur les *runs* où elle est activée.

4.3 Vue d'ensemble de l'architecture

4.3.1 Topologie en huit agents

Le pipeline se compose de huit agents principaux organisés en pipeline avec branchements dynamiques :

A1 — Orchestrateur

supervise l'exécution, initialise l'état, produit le rapport final.

A2 — Analyste

extraie de la fiche libre un profil de compétences structuré (*hard skills*, *soft skills*, séniorité, contexte).

A3a — Stratège

(sous-agent de recherche) génère un plan de requêtes adaptées par source.

A3b — Collecteur

exécute les requêtes sur les sources gratuites (DuckDuckGo, GitHub, Stack Exchange).

A3c — Filtre anti-bruit

applique un filtrage algorithmique des hits (domaines bloqués, mots-clés, URL).

A6 — Déduplicateur

fusionne les profils redondants par similarité de noms et d’URL.

A4 — Évaluateur

(instances parallèles) note chaque candidat sur la base de la fiche.

A5 — Vérificateur

croise les scores et les profils bruts pour invalider ou ajuster.

A7 — Recruteur

rédige les messages de contact, après validation humaine.

A8 — Persistance RAG

stocke fiche et candidats validés en mémoire vectorielle.

La numérotation A1 à A8 correspond à la séquence logique vue par le recruteur, sans correspondre strictement à l’ordre d’exécution ; A3a/b/c sont trois sous-agents d’A3, et A4 est instancié N fois en parallèle (un par candidat).

4.3.2 Flux du pipeline

La figure 4.2 donne une vue complète du flux. Le pipeline progresse séquentiellement de A1 jusqu’à A6, puis s’éclate en N instances d’A4 (un par candidat), converge dans un *reducer*, est validé par A5, est routé par seuil de score, passe éventuellement par un point d’arrêt humain (HITL) avant A7, et s’achève par la génération du rapport et la persistance RAG par A8.

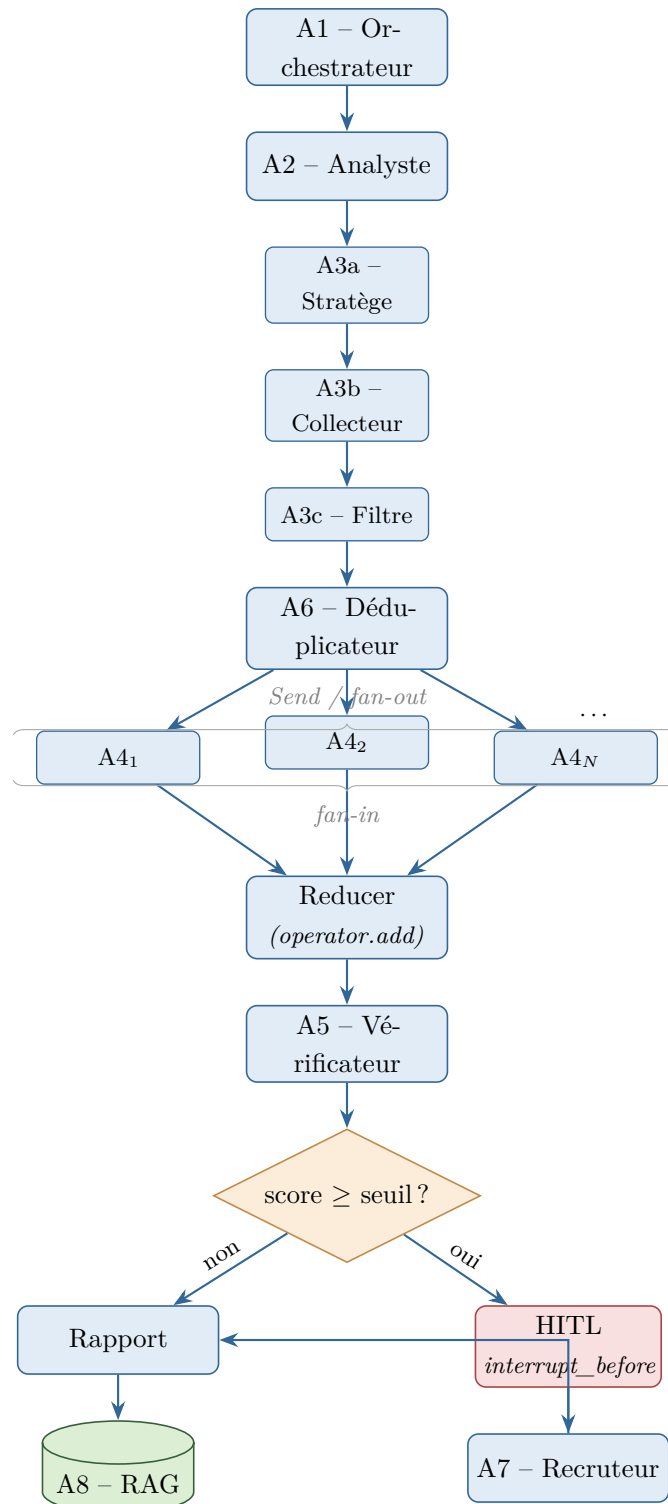


FIGURE 4.2 – Pipeline global du système. A1–A2 puis A3a/b/c constituent la phase de cadrage et de *sourcing*. A6 déduplique. A4 est instancié N fois en parallèle (*Send*), un *reducer* agrège les scores, A5 valide. Le routage par seuil dirige le flux vers A7 (sous interruption HITL) ou directement vers le rapport. A8 persiste fiche et candidats validés en RAG.

Le routage après A5 est conditionnel : si le meilleur candidat dépasse un seuil absolu, ou si le top-N relatif comporte des candidats viables, le flux passe par A7 (Recruteur, sous interruption HITL). Sinon, il passe directement au rapport. Dans tous les cas, A8 (persistance RAG) clôt

l’exécution.

4.3.3 État partagé

Tous les agents communiquent via un état partagé typé `GraphState` (`TypedDict` dans `src/state.py`).

Les principaux champs sont :

- `fiche_poste` : texte libre fourni en entrée ;
- `profil_competences` : structure produite par A2 ;
- `requetes_recherche` : plan de requêtes produit par A3a ;
- `profils_bruts` : hits collectés et filtrés ;
- `profils_dedupliques` : profils consolidés par A6 ;
- `candidats_scores` : scores produits par les instances d’A4 ;
- `candidats_valides` : sortie de A5 (avec statuts *validé*, *douteux*, *invalide*) ;
- `messages_envoyes` : messages rédigés par A7 ;
- `rapport` : rapport final produit par A1 ;
- `metriques` : journal d’observabilité.

Les champs lus/écrits par plusieurs agents en parallèle sont annotés `Annotated[list, operator.add]` pour garantir la cohérence des écritures concurrentes. La figure 4.3 en donne une vue UML simplifiée, accompagnée des structures `Candidat`, `CandidatScore` et `CandidatValide` produites successivement par les agents, ainsi que des services transverses `MemoireRAG` et `PipelineMetrics`.

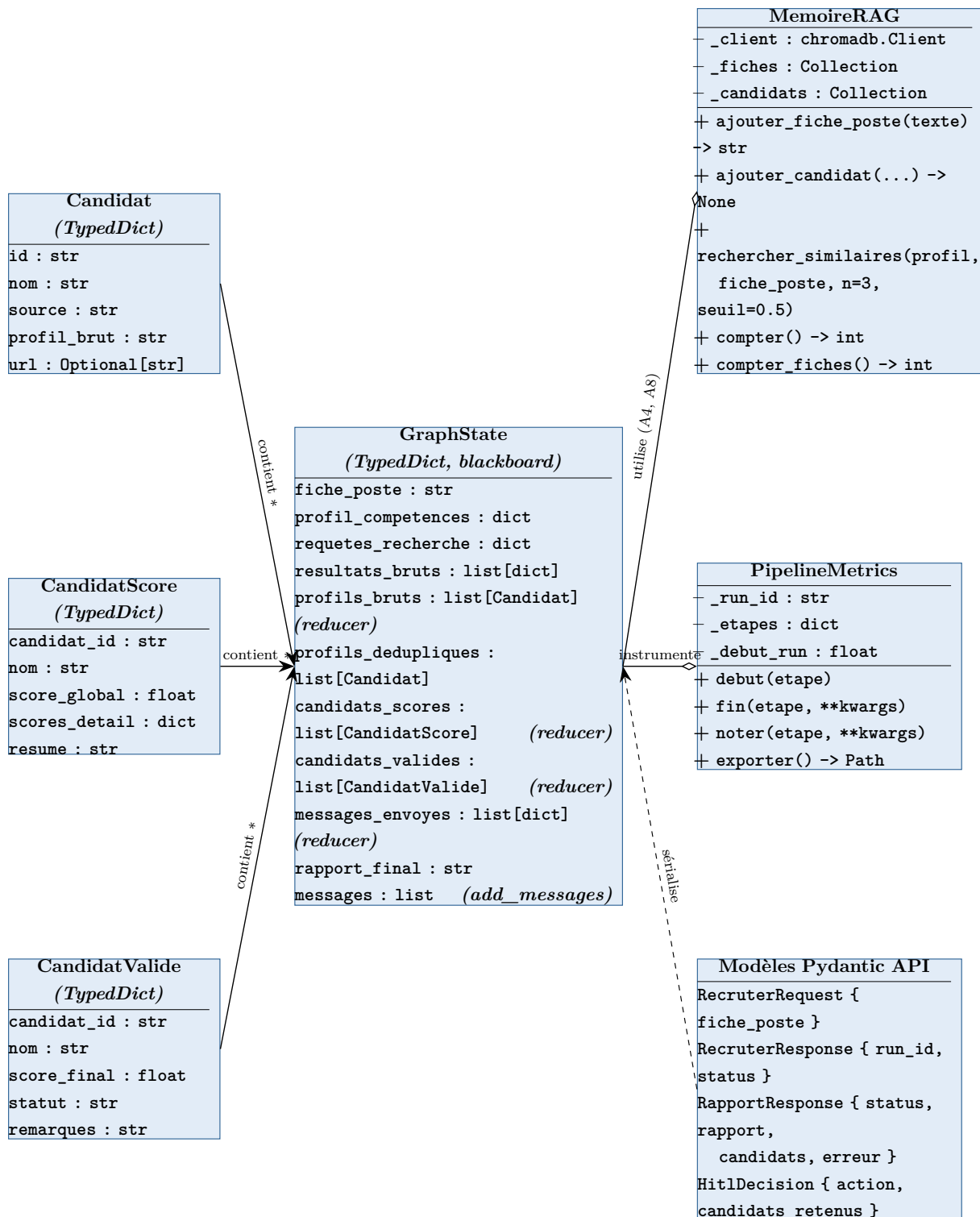


FIGURE 4.3 – Diagramme de classes simplifié. **GraphState** occupe la position centrale du tableau noir SMA, agrégeant les structures **Candidat**, **CandidatScore** et **CandidatValide** produites successivement par les agents. **MemoireRAG** et **PipelineMetrics** sont des services transverses ; les modèles Pydantic exposent **GraphState** en JSON via l'API REST.

4.3.4 Stack technique

Couche	Technologie retenue
Orchestration multi-agents	LangGraph [19]
Modèle de langage	Ollama (<i>kimi-k2.5 :cloud</i> par défaut) [24]
<i>Embeddings</i>	SentenceTransformers <i>all-MiniLM-L6-v2</i> [25]
Mémoire vectorielle	ChromaDB [26]
Recherche web	DuckDuckGo (<i>duckduckgo-search</i>)
Recherche profils	GitHub Search Users API, Stack Exchange API
<i>Scraping</i>	<i>requests</i> + BeautifulSoup 4
API Gateway	FastAPI + Uvicorn [27]
Streaming temps réel	Server-Sent Events [28]
Interface graphique	NiceGUI [29]
Conteneurisation	Docker & Docker Compose
Tests automatisés	<i>pytest</i>

TABLE 4.1 – Stack technique du système. Chaque choix est justifié au chapitre 5.

La figure 4.4 synthétise l'architecture en composants : trois couches (présentation, service, orchestration) et trois capacités mobilisées (recherche, mémoire vectorielle, LLM).

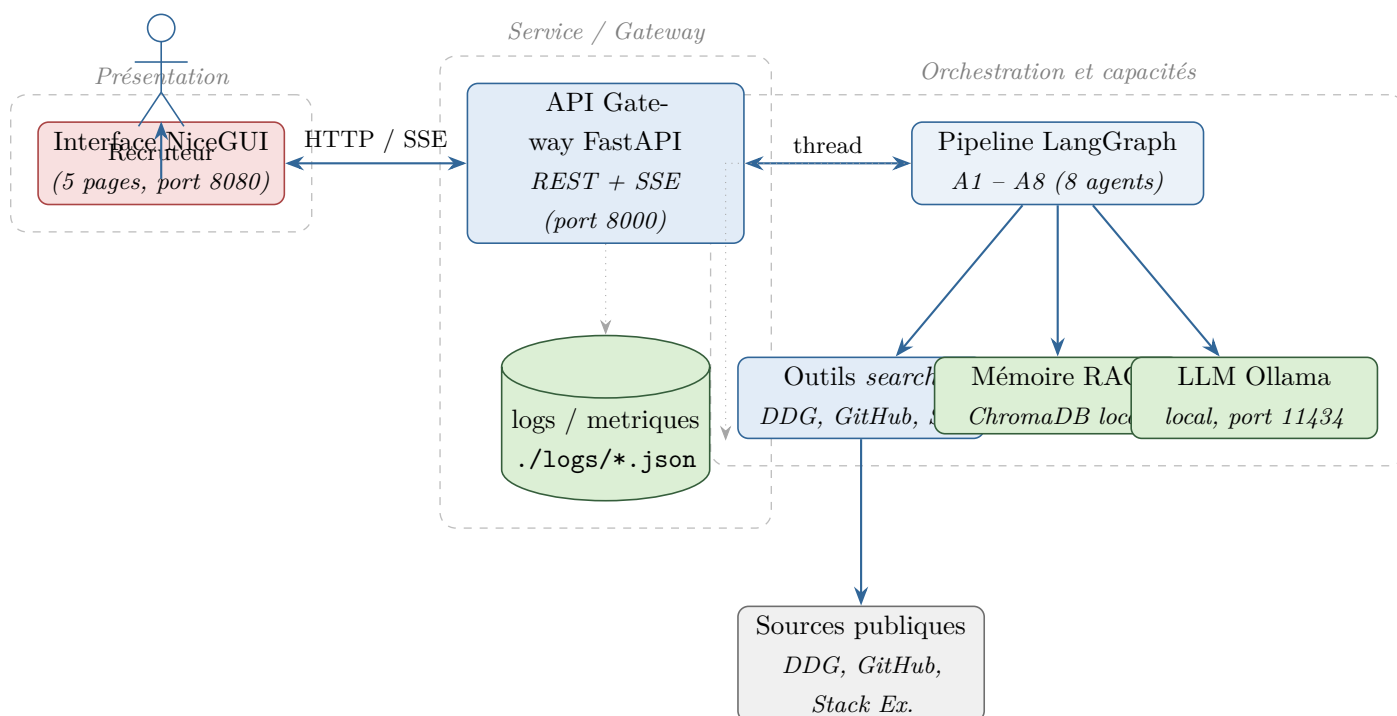


FIGURE 4.4 – Architecture en composants de l'application. Trois couches successives séparent la présentation (NiceGUI), le service (API Gateway FastAPI) et l'orchestration (LangGraph). Les capacités externes (recherche web, mémoire vectorielle, LLM) sont mobilisées par le pipeline de manière encapsulée. Les exports d'observabilité passent par le service.

4.4 Périmètre du système

4.4.1 Inclus dans le périmètre

- Sourcing automatique sur trois sources gratuites (DDG, GitHub, Stack Exchange).
- Évaluation par LLM local avec scoring multi-axes et explications.
- Vérification pair-à-pair indépendante (A5).
- Validation humaine explicite avant tout contact effectif (HITL).
- Mémoire vectorielle calibrée par fiche de poste.
- Interface graphique avec suivi temps réel et exploration de la mémoire.
- Observabilité : journaux structurés, exports JSON, page *métriques*.
- Tests automatisés sur les composants critiques.

4.4.2 Hors du périmètre

- Sourcing sur LinkedIn, Malt, Indeed (refusé pour des raisons RGPD, ToS et risque juridique). Discuté en section 3.5.
- Envoi effectif de messages : le système rédige et propose, mais n'envoie pas. Le recruteur valide et exécute manuellement, ce qui préserve la traçabilité humaine.
- Apprentissage en ligne : aucun *fine-tuning* du LLM, aucun apprentissage par renforcement entre agents. Le RAG calibré constitue le seul mécanisme de mémoire inter-runs.
- Multi-tenant : le système n'est pas conçu pour servir plusieurs recruteurs simultanément avec des données isolées par tenant. Cette extension est laissée aux perspectives.
- Intégration ATS (*Applicant Tracking System*) : aucun connecteur vers Workday, SAP SuccessFactors ou équivalents. La sortie reste à la main du recruteur.

L'analyse détaillée de chaque composant et de chaque choix d'implémentation fait l'objet du chapitre suivant.

Chapitre 5

Analyse du système

Ce chapitre analyse en profondeur l’implémentation du système. L’architecture se prêtant naturellement à un découpage en trois pans, le chapitre est structuré en trois sous-chapitres :

- § 5.A Le **cœur LangGraph** : topologie, état partagé, patrons de coordination, choix structurants (`StateGraph`, *Send/Reduce*, `interrupt_before`, validation pair-à-pair, choix Ollama).
- § 5.B Les **outils et la mémoire RAG** : pipeline de *sourcing* A3a/b/c, filtre anti-bruit algorithmique, mémoire vectorielle calibrée par fiche.
- § 5.C La **couche service et opérations** : API Gateway FastAPI, streaming SSE, signalement HITL par `threading.Event`, interface graphique NiceGUI, observabilité, tests, déploiement Docker.

Chaque sous-chapitre suit la même grille de lecture : présentation des composants, justification argumentée des choix d’implémentation (besoin / alternatives / décision / compromis assumés), et extraits de code commentés. Les choix transversaux (LangGraph, Ollama, ChromaDB, NiceGUI, sources gratuites) sont rappelés et reliés aux argumentaires théoriques du chapitre 2 et à la décision de *framework* du chapitre 3.

5.1 Architecture du graphe LangGraph

Le système de recrutement implémente un superviseur décentralisé via un `StateGraph` LangGraph : une machine à états typée où l’état global `GraphState` (voir `src/state.py`) joue le rôle de tableau noir (*blackboard*) au sens des SMA classiques. Cet état comprend une quinzaine de champs distincts, chacun dédié en écriture à un agent ou à un groupe d’agents : orchestrateur → analyste → pipeline A3 (stratège → collecteur → filtre) → déduplicateur → évaluateurs parallèles → vérificateur → recruteur ou rapport → persistance RAG.

Les nœuds du graphe compilé (`src/graph.py`) forment une topologie en pipeline avec deux branchements décisionnels significatifs : (i) un *fan-out* vers *N* évaluateurs parallèles au départ de chaque profil déduplicué, déclenché par un `Send()` ; (ii) un *fan-in* par agrégation des scores via un nœud de réduction explicite ; (iii) un routage conditionnel après vérification, qui aiguille soit

vers le recruteur, soit vers le rapport en fonction des seuils de score. L'ordre d'exécution respecte une dépendance stricte : huit nœuds séquentiels (A1 à A8) plus les instances parallèles d'A4.

Les champs du `GraphState` typés `Annotated[list, operator.add]` (`profils_bruts`, `candidats_scores`, `messages_envoyes`) implémentent des *reducers* qui agrègent les résultats des instances parallèles sans conflit. Le compilateur `LangGraph` synchronise automatiquement les N branches d'évaluation sur le nœud de réduction, garantissant la cohérence des écritures concurrentes sur l'état partagé.

5.2 Patterns de coordination retenus

Cinq patrons de coordination sont mobilisés dans le graphe, chacun directement transposable à la terminologie SMA classique.

Send/Map-Reduce (fan-out/fan-in). La fonction `route_to_evaluateurs()` (voir `src/graph.py`) crée une liste de N appels `Send()`, chacun instanciant une copie d'A4 (Évaluateur) avec un candidat distinct et le profil de compétences partagé. Le *reducer* `operator.add` fusionne les listes `candidats_scores` produites par chaque instance. La constante `MAX_PROFILS_PARALLELES` (10 par défaut, dans `src/config.py`) borne la concurrence afin d'éviter la saturation de l'API LLM locale (Ollama). Le nœud `reduce_scores` synchronise la réduction et journalise les statistiques consolidées. La figure 5.1 illustre ce patron.

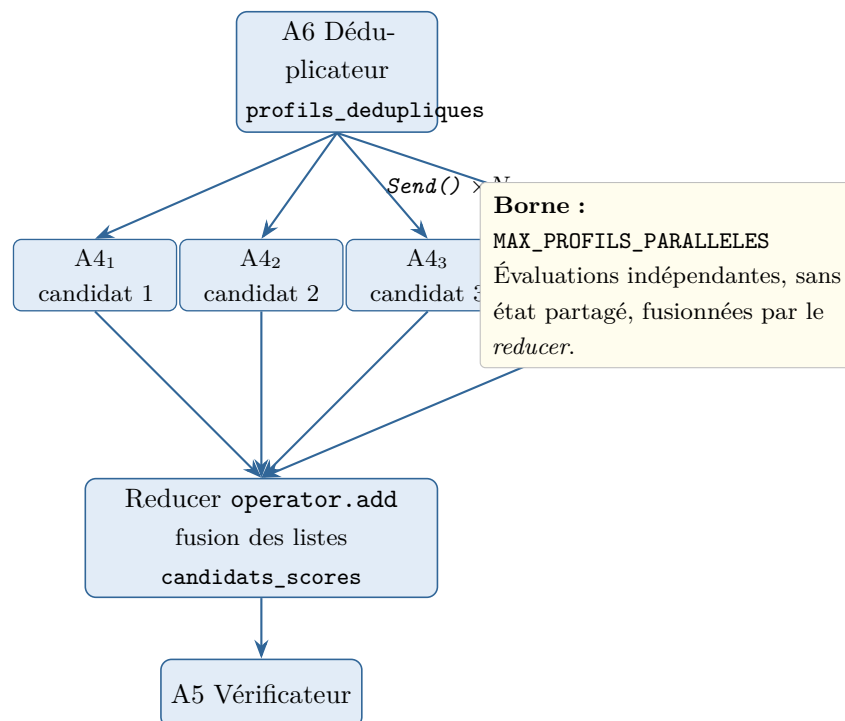


FIGURE 5.1 – Pattern *Send / Reduce* de LangGraph appliqué à l'évaluation parallèle. Chaque instance d'A4 traite un candidat distinct ; le *reducer* fusionne les listes `candidats_scores` sans collision.

Validation pair-à-pair (A4 → A5). L'évaluateur (A4) produit des scores bruts dans le champ `candidats_scores`; le vérificateur (A5) reçoit ces scores et croise avec les profils bruts conservés en `profils_dedupliques`, puis valide ou invalide chaque candidat indépendamment, en produisant `candidats_valides` avec ajustements ou statut « douteux ». Cette séparation formelle des champs impose une vérification transversale obligatoire : A5 contrôle la cohérence intra-CV (dates suspectes, compétences invraisemblables) et inter-candidats (signaux d'alerte, dérives de scoring) sans que le résultat brut d'A4 ne puisse être implicitement réutilisé.

Routage conditionnel par score. La fonction `route_apres_verification()` (`src/graph.py`) applique une logique à deux étages. Si le meilleur candidat dépasse `SCORE_SEUIL_CONTACT` (75 par défaut), le flux est dirigé vers A7 (Recruteur). Sinon, si au moins un candidat dépasse `SCORE_SEUIL_VIABLE` (40), les `TOP_N_RELATIF` meilleurs (3 par défaut) passent en « mode relatif » via A7 avec signalisation explicite. Au-delà, le flux est dirigé directement vers le rapport. Les seuils sont configurables via variables d'environnement.

Interrupt_before pour le human-in-the-loop. Lors de la compilation du graphe, l'option `interrupt_before=["recruteur"]` est appliquée au *checkpoint*. Cela suspend l'exécution juste avant le nœud A7, permettant à un humain de valider, d'éditer ou de rejeter la liste de candidats avant tout contact automatisé. La reprise est faite via `app_graph.update_state()` puis `invoke(None, config)` : le graphe redémarre exactement à l'endroit où il s'était arrêté.

Mémoire contextuelle (RAG + Persistance). Le nœud A8 (`src/agents/persistence.py`) stocke chaque fiche de poste et ses candidats validés dans ChromaDB, reliés par un `fiche_id` stable. L'évaluateur A4 interroge cette mémoire en début de scoring, récupérant les profils similaires évalués pour des postes comparables à titre de calibration inter-runs.

5.3 Justification des choix d'implémentation

5.3.1 StateGraph et état typé plutôt que MessageGraph

Besoin. Coordonner huit agents spécialisés produisant des artefacts hétérogènes (listes de requêtes, hits bruts, profils dédupliques, scores détaillés, validations, messages, rapport), avec dépendances complexes et exigence d'analyse *post mortem*.

Alternatives. Une approche `MessageGraph` (style chat multi-tour) accumulerait tous les résultats dans une liste `messages` de `LangChain` : pratique pour le débogage de prototypes conversationnels, mais sérialisation volumineuse, absence de typage métier, fusion manuelle des résultats parallèles, routage conditionnel maladroît.

Décision. `StateGraph` avec `TypedDict` fortement typé. Chaque champ de `GraphState` documente qui écrit et qui lit. Les *reducers* `operator.add` sur les champs parallèles éliminent les

collisions de fusion. Les nœuds retournent des dictionnaires partiels, fusionnés automatiquement par le compilateur LangGraph. Le typage statique détecte les fautes de champ dès le développement.

Tradeoffs. Approche moins flexible qu’un journal de messages plat pour explorer du code dynamique; exige une architecture bien définie en amont. Compromis assumé : huit agents spécialisés ne forment pas un système conversationnel.

5.3.2 Send/Reduce pour l’évaluation parallèle

Besoin. Chaque candidat doit être évalué indépendamment (LLM sans état partagé). L’évaluation séquentielle d’un lot de dix profils prendrait de l’ordre de 30 à 40 secondes; en parallèle, trois à quatre secondes suffisent.

Alternatives. (i) Boucle interne dans un nœud monolithique; (ii) orchestration externe à base de threads Python; (iii) implémentation manuelle d’un *map-reduce* avec fusion ad hoc.

Décision. Le patron `Send()` de LangGraph crée implicitement N instances légères du nœud évaluateur, chacune traitant un candidat isolé, avec synchronisation et fusion automatiques. Le `reducer operator.add` agrège les listes sans logique de remplacement.

Tradeoffs. La concurrence est bornée par `MAX_PROFILS_PARALLELES` pour éviter la surcharge d’Ollama local; les batches successifs résolvent le cas où plus de candidats doivent être évalués.

5.3.3 Interrupt_before sur A7

Besoin. Avant tout envoi de message de contact, un humain doit valider la pertinence de la sélection et le contenu rédigé.

Alternatives. (i) Pause externe gérée par l’API; (ii) nœud A7 demandant un *input* synchrone; (iii) absence de pause, avec audit *post mortem*.

Décision. L’option `interrupt_before` du compilateur LangGraph arrête l’exécution juste avant A7. L’état est entièrement accessible, modifiable et rejouable via le *checkpoint*.

Tradeoffs. Étape bloquante potentiellement longue; la transparence et la réduction des erreurs irréversibles justifient le coût dans un contexte RH.

5.3.4 Routage conditionnel à seuils absolu et relatif

Besoin. Distinguer deux scénarios opérationnels : (i) au moins un candidat clairement au-dessus du seuil (contact automatique); (ii) aucun excellent, mais quelques viables (top-N relatif soumis

à validation humaine).

Alternatives. Toujours contacter le top-1 ; contacter tous les candidats au-dessus d'un seuil unique ; deux graphes séparés.

Décision. Routage dual intégré. `route_apres_verification()` distingue les deux modes et bascule A7 ou rapport en conséquence. Les seuils `SCORE_SEUIL_CONTACT`, `SCORE_SEUIL_VIABLE` et `TOP_N_RELATIF` sont configurables par variables d'environnement.

Tradeoffs. Logique métier dans le graphe ; justifié par la diversité des fiches de poste et l'asymétrie des marchés selon le métier.

5.3.5 Séparation A4 (Évaluateur) / A5 (Vérificateur)

Besoin. L'évaluation par LLM est probabiliste (hallucinations, biais de *prompt*, dérives de scoring) ; un contrôle transversal indépendant est nécessaire.

Alternatives. A4 unique avec auto-vérification ; boucle de *feedback* $A4 \rightarrow A5 \rightarrow A4$ récursive.

Décision. Deux agents distincts avec champs d'état séparés. A5 reçoit les scores et les profils bruts originaux, et vérifie incohérences et signaux suspects. Patron de *checks and balances* formalisé en SMA classique.

Tradeoffs. Doublement des appels LLM (latence accrue) ; justifié par la criticité RH d'un mauvais scoring.

5.3.6 Ollama local plutôt que API externe (Anthropic, OpenAI)

Besoin. Souveraineté des données candidats, coût marginal nul après déploiement, démonstration hors ligne, conformité RGPD.

Alternatives. Claude API (qualité premium, coût et exposition externe des PII) ; GPT-4 (idem) ; LLaMA local (qualité moindre sur le raisonnement complexe).

Décision. Ollama via `init_chat_model` de `langchain_classic`, avec modèle local configurable (`OLLAMA_MODEL=kimi-k2.5:cloud` par défaut). Le fournisseur Ollama est abstrait par LangChain, facilitant un éventuel basculement vers Anthropic ou OpenAI.

Tradeoffs. Qualité de raisonnement légèrement inférieure aux modèles *frontier*, atténuée par un *prompt engineering* contraint, une température basse (0 pour l'évaluation, 0,7 pour la rédaction) et une structuration JSON stricte des réponses.

5.3.7 Découpage en huit agents spécialisés

Besoin. Modéliser six phases métier (analyse de fiche, sourcing, déduplication, évaluation, vérification, contact, rapport) plus une phase technique (persistance RAG).

Alternatives. Monolithe unique ; trois ou quatre agents génériques ; douze sous-agents très spécialisés.

Décision. Un agent par phase. A1 (superviseur), A2 (analyse), A3a/b/c (sourcing décomposé), A4, A5, A6, A7, A8. Chaque agent lit un sous-ensemble précis du **GraphState**, écrit ses résultats, et ignore les autres.

Tradeoffs. Davantage de champs d'état à maintenir ; compense par la testabilité, la traçabilité (logs par agent) et la maintenabilité (modifier un agent ne nécessite pas de refactor systémique).

5.3.8 Checkpointer (MemorySaver)

Besoin. Reprendre l'exécution après interruption HITL ou crash, et exporter l'historique pour audit.

Alternatives. Absence de *checkpoint* ; base externe (PostgreSQL) ; sérialisation ad hoc.

Décision. Le compilateur LangGraph reçoit `checkpointer=MemorySaver()`. Les snapshots d'état sont stockés en mémoire (suffisant pour le démonstrateur). Une variante `SqliteSaver` permet la persistance entre sessions sans changement de code applicatif.

Tradeoffs. `MemorySaver` perd l'historique au redémarrage du processus ; `SqliteSaver` le persiste mais introduit un fichier de base local.

5.4 Extraits commentés

```

1 def route_to_evaluateurs(state: GraphState) -> list[Send]:
2     """Fan-out : envoie chaque profil déduplicué à une instance d'évaluateur.
3     """
4     profils = state.get("profils_dedupliques", [])
5     if not profils:
6         return [Send("reduce_scores", {})]
7
8     if len(profils) > MAX_PROFILS_PARALLELES:
9         profils = profils[:MAX_PROFILS_PARALLELES]
10
11     return [
12         Send("evaluator", {

```

```

12         "candidat": candidat,
13         "profil_competences": state["profil_competences"],
14         "fiche_poste": state.get("fiche_poste", ""),
15     })
16     for candidat in profils
17 ]

```

Listing 5.1 – Création des instances parallèles d'évaluateur via `Send()` (`src/graph.py`)

Ce patron `Send()` instancie N nœuds `evaluator` isolés, chacun recevant un candidat distinct et les données partagées. `LangGraph` lance ces N branches en parallèle, attend leur convergence et fusionne les résultats via le `reducer operator.add`.

```

1 def route_apres_verification(state: GraphState) -> str:
2     """Routage après vérification : absolu si meilleur >= seuil, sinon relatif"""
3     candidats_valides = state.get("candidats_valides", [])
4     if not candidats_valides:
5         return "rapport"
6
7     meilleur_score = max(c["score_final"] for c in candidats_valides)
8
9     if meilleur_score >= SCORE_SEUIL_CONTACT:
10        return "recruteur"
11
12    viables = [c for c in candidats_valides
13               if c["score_final"] >= SCORE_SEUIL_VIABLE]
14    if viables:
15        return "recruteur"    # Top-N relatif
16
17    return "rapport"

```

Listing 5.2 – Routage conditionnel absolu / relatif (`src/graph.py`)

La logique à deux étages bascule entre contact automatique (mode absolu, seuil strict), top-N relatif (si aucun excellent mais quelques viables) ou rapport sans contact. Les seuils sont configurables.

```

1 def verificateur_node(state: GraphState) -> dict:
2     """Vérifie les scores de A4 : incohérences dates, CV gonflés, signaux fraude."""
3     candidats_scores = state.get("candidats_scores", [])
4     llm = init_chat_model(OLLAMA_MODEL, model_provider=OLLAMA_PROVIDER,
5                           temperature=0)
6
7     profils_par_id = {p["id"]: p for p in state.get("profils_dedupliques", [])}
8
9     candidats_info = []
10    for cs in candidats_scores:
11        entry = {
12            "candidat_id": cs["candidat_id"],
13            "nom": cs["nom"],

```

```

12         "score_global": cs["score_global"],
13         "scores_detail": cs["scores_detail"],
14         "resume_evaluateur": cs["resume"],
15     }
16     profil_brut = profils_par_id.get(cs["candidat_id"])
17     if profil_brut:
18         entry["profil_brut"] = profil_brut["profil_brut"]
19         candidats_info.append(entry)
20     # le LLM ré-évalue : cohérence des dates, plausibilité des compétences,
21     # détection d'artefacts suspects.

```

Listing 5.3 – Validation pair-à-pair : A5 ré-inspecte avec les profils bruts (src/agents/verificateur.py)

A5 reçoit les scores produits par A4 et les profils bruts originaux conservés dans un champ distinct du `GraphState`, ce qui rend possible une vérification transversale des incohérences (dates suspectes, titres vagues, technologies incompatibles). La séparation explicite des champs rend cette validation obligatoire.

```

1  def persistence_node(state: GraphState) -> dict:
2      """Stocke fiche + candidats validés en ChromaDB pour RAG futur."""
3      candidats_valides = state.get("candidats_valides", [])
4      fiche_poste = state.get("fiche_poste", "")
5
6      memoire = get_memoire()
7
8      fiche_id = memoire.ajouter_fiche_poste(fiche_poste)
9
10     profils_idx = {p["id"]: p for p in state.get("profils_dedupliques", [])}
11
12     for candidat in candidats_valides:
13         if candidat.get("statut") == "invalid":
14             continue # Ne pas persister le bruit
15
16         profil = profils_idx.get(candidat["candidat_id"], {})
17         memoire.ajouter_candidat(
18             candidat_id=candidat["candidat_id"],
19             nom=candidat["nom"],
20             profil_brut=profil.get("profil_brut", "..."),
21             score=candidat["score_final"],
22             fiche_id=fiche_id, # lien stable pour requête RAG ultérieure
23         )

```

Listing 5.4 – Persistance RAG et lien fiche ↔ candidat (src/agents/persistence.py)

A8 constitue le maillon de mémoire : chaque fiche de poste reçoit un identifiant stable (hash du contenu), puis chaque candidat validé est indexé en ChromaDB avec ce `fiche_id`. Les exécutions futures peuvent ainsi retrouver les contextes pertinents via `rechercher_similaires()` dans A4.

5.5 Pipeline de sourcing multi-sources (A3)

Le pipeline de collecte de candidats (agent A3) se décompose en trois sous-agents spécialisés, conformément au patron de séparation des préoccupations. Cette décomposition répond à deux objectifs : (i) clarifier les responsabilités cognitives (*que chercher ?*), logistiques (*comment chercher ?*) et décisionnelles (*quoi conserver ?*) ; (ii) permettre à chaque couche de fonctionner indépendamment et d'être substituée sans répercussion sur les autres.

A3a — Stratège (src/agents/chercheur_strategie.py). Le stratège reçoit le profil de compétences extrait par A2 et produit un plan de recherche structuré. Il invoque le LLM avec un *prompt* imposant quatre familles de requêtes optimisées par source : requêtes web générales (avec opérateurs comme `intitle:CV`), requêtes LinkedIn ciblées (`site:linkedin.com/in`), requêtes natives pour l'API GitHub (par exemple `language:Python location:Paris followers:>5`) et requêtes pour agrégateurs de portfolios (Malt, DoYouBuzz). Le *prompt* impose également le respect strict du niveau d'expérience : pour les niveaux « alternant » ou « stagiaire », des exclusions `-"senior"` `-"lead"` sont ajoutées en post-traitement. Le stratège produit un dictionnaire `requetes_recherche` avec les clés `queries_generales`, `queries_linkedin`, `queries_github`, `queries_cv_sites` et `tags_stackoverflow`. Aucun appel réseau n'est effectué à ce stade : le LLM génère les requêtes de manière déclarative.

A3b — Collecteur (src/agents/chercheur_collecteur.py). Le collecteur exécute les requêtes générées par le stratège sur cinq sources gratuites :

1. DuckDuckGo pour les requêtes web générales, LinkedIn et portfolios ;
2. l'API officielle GitHub Search Users (60 requêtes par heure sans *token*, 5 000 avec un *token* gratuit) ;
3. l'API Stack Exchange (300 requêtes par jour sans clé, 10 000 avec clé gratuite).

DuckDuckGo est utilisé comme moteur généraliste avec des opérateurs proches des moteurs classiques : `site:`, guillemets exacts et exclusions `-"mot"`. Les recherches LinkedIn ciblent donc les pages de profils publiques avec `site:linkedin.com/in` ou `site:fr.linkedin.com/in`, et non `linkedin.fr`, qui n'est pas le domaine usuel des profils publics. Chaque source retourne des résultats structurés au format unifié `{title, url, body, source}`. Le collecteur agrège l'ensemble, puis effectue une première déduplication par URL. Aucune décision algorithmique ou linguistique n'est appliquée à ce stade : le nœud reste *stateless* et déterministe. Les journaux reportent le nombre de résultats bruts par source, ce qui facilite l'observabilité du bruit amont.

A3c — Filtre (src/agents/chercheur_filtre.py). Le filtre transforme les hits bruts en profils candidats exploitables via un filtrage algorithmique multi-étapes, sans recours au LLM (voir section suivante). Les profils générés sont limités à `MAX_PROFILS_RECHERCHE` (15 par défaut). Le filtre consigne trois métriques : taux de bruit brut (hits éliminés / hits collectés), nombre d'échecs de *scraping* (repli sur le *snippet* DuckDuckGo) et nombre final de profils exploitables.

La séquence complète suit un flux LangGraph linéaire $A3a \rightarrow A3b \rightarrow A3c$, représenté figure 5.2. Le nœud A3c produit une liste `profils_bruts` prête à être consommée par le déduplicateur (A6) puis l'évaluateur (A4).

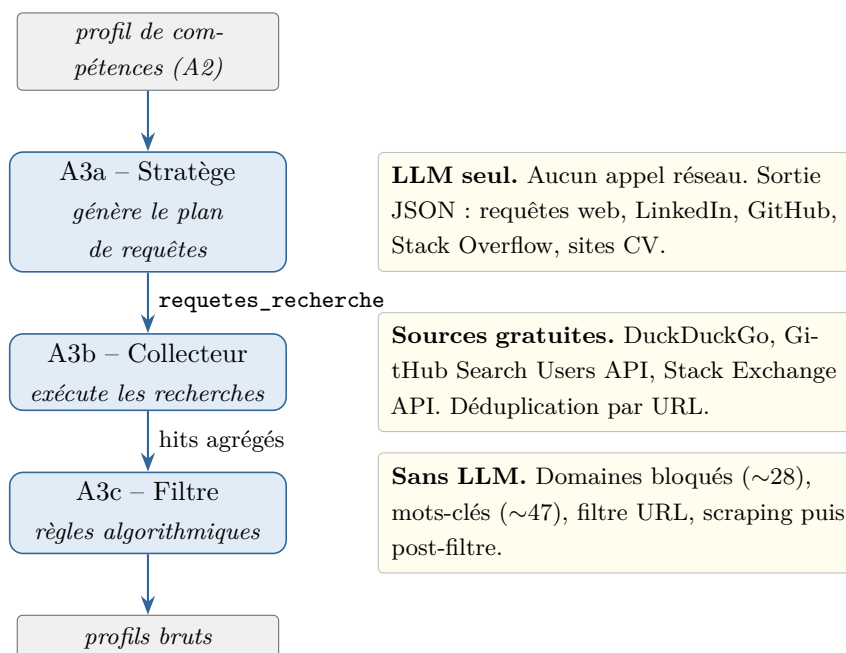


FIGURE 5.2 – Pipeline de *sourcing* A3 en trois étapes spécialisées. Le stratège est cognitif (LLM), le collecteur est purement logistique, le filtre est strictement algorithmique.

5.6 Filtre anti-bruit algorithmique

Le filtre d'A3c incarne un choix délibéré : appliquer des règles algorithmiques déterministes plutôt qu'un filtrage par LLM. Trois arguments justifient cette décision.

Coût et latence. Un filtrage par LLM supposerait d'invoquer le modèle pour chaque hit brut, amplifiant la latence globale du pipeline (déjà 2 à 5 minutes) et augmentant la charge de calcul. Avec une moyenne de 40 à 50 hits par recherche avant filtrage et 5 à 8 requêtes par source, cela représenterait 200 à 400 appels LLM pour une seule exécution. Les règles algorithmiques (domaines bloqués, mots-clés, fragments d'URL) sont au contraire exécutées en environ une milliseconde par hit.

Déterminisme et auditabilité. Les listes `_NOISE_DOMAINS` et `_NOISE_KEYWORDS` sont codifiées en clair, lisibles et modifiables sans réentraînement. Un évaluateur du rapport peut consulter le code source et comprendre exactement quels domaines et termes sont exclus. Un filtre LLM serait au contraire une boîte noire probabiliste : deux exécutions sur les mêmes données pourraient produire des décisions différentes.

Stabilité face au bruit. Les domaines d'agréateurs et les formulations « offre d'emploi » sont stables dans le temps et largement universels. Un LLM, entraîné sur une distribution générale

d’Internet, peut être confus par des cas limites (un profil LinkedIn dont la description évoque le mot « candidature ») ou dévier si la température est élevée. Les règles algorithmiques restent insensibles à ces variations.

Limitations acceptées. Le filtre ne détecte pas le bruit sémantique fin (CV générique, profil copié d’un agrégateur). Ce résidu est absorbé en aval par l’évaluateur A4, qui s’appuie sur la similarité sémantique via le RAG et le jugement du LLM. Cet arbitrage entre filtrage précoce et filtrage tardif est documenté dans les journaux : un taux de suppression de 40 à 60 % des hits bruts est observé en pratique.

5.7 Mémoire RAG calibrée par fiche de poste

La mémoire vectorielle (`src/tools/rag.py`, classe `MemoireRAG`) résout un problème structurel du RAG naïf inter-fiches. Dans un système de recrutement multi-runs, chaque candidat reçoit un score d’évaluation dans le contexte d’une fiche de poste donnée. Réutiliser ce score comme signal de calibration pour une autre fiche est trompeur : un candidat évalué 85/100 pour « Développeur Python senior » n’est pas pertinent comme référence pour « Chef de projet marketing », même en cas de similarité textuelle superficielle.

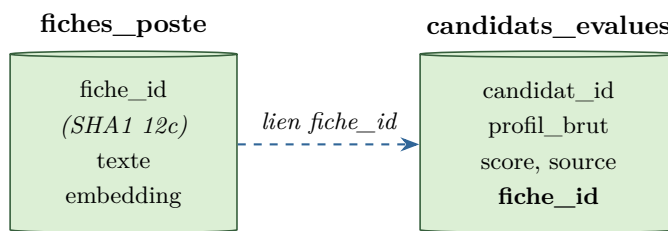
5.7.1 Architecture à deux collections

La base ChromaDB comporte deux collections reliées par un identifiant stable.

Collection	Document	Métadonnées
<code>fiches_poste</code>	texte de la fiche (3 000 caractères max.)	longueur
<code>candidats_evalues</code>	<code>profil_brut</code> (2 000 caractères max.)	nom, score, source, <code>fiche_id</code>

TABLE 5.1 – Collections ChromaDB et leurs métadonnées. Le champ `fiche_id` relie chaque candidat à la fiche pour laquelle il a été évalué.

L’identifiant `fiche_id` est défini par `hash_fiche(texte)`, un SHA1 tronqué calculé sur le texte normalisé (espaces compressés, casse unifiée). Le hash est déterministe : deux textes identiques au formatage près produisent le même identifiant, garantissant l’idempotence des *upserts*. L’ensemble est représenté figure 5.3.



Procédure d'interrogation (A4 lors du scoring d'un nouveau candidat) :

1. *embed* la fiche courante ;
2. interroger **fiches_poste**, retenir les fiches dont la similarité cosinus $\geq 0,5$;
3. si aucune n'atteint le seuil $\rightarrow$ retour `[]` (silence explicite) ;
4. sinon, interroger **candidats_evalues** avec `where={fiche_id : $in : [...] }`.

FIGURE 5.3 – Architecture de la mémoire RAG calibrée par fiche de poste. Les deux collections sont reliées par un identifiant stable **fiche_id** qui filtre la recherche de candidats sur les fiches sémantiquement proches, écartant le biais inter-fiches du RAG naïf.

5.7.2 Procédure de lecture en deux temps

Lorsque A4 évalue un nouveau candidat pour une fiche courante, l'agent appelle `rechercher_similaires(profil fiche_poste=...)`. Le code effectue alors :

1. un *embedding* de la fiche courante via `SentenceTransformer all-MiniLM-L6-v2` ;
2. une interrogation de **fiches_poste** avec un seuil de similarité cosinus $\geq 0,5$;
3. si aucune fiche stockée ne dépasse le seuil, la fonction retourne une liste vide *sans injecter de contexte RAG* ;
4. sinon, une interrogation de **candidats_evalues** avec le filtre `where={"fiche_id": {"$in": [...]}}`, qui ne remonte que les candidats rattachés à des fiches similaires.

Cette procédure élimine le biais inter-fiches tout en maintenant un silence explicite (aucun contexte injecté) sur une fiche inédite. Au premier *run*, le RAG ne produit rien ; sur le second *run* avec une fiche analogue, les candidats du premier deviennent une référence de calibration valide.

5.7.3 Garanties testées

Trois invariants sont vérifiés unitairement (voir `tests/test_rag.py`, classe `TestCalibrationParFiche`) :

1. **Hash stable** : deux textes identiques aux espaces et à la casse près produisent le même `fiche_id`.
2. **Isolation inter-fiches** : si deux fiches très différentes coexistent en base et que la fiche courante ressemble à la première, seuls les candidats de la première remontent.
3. **Repli silencieux** : si aucune fiche stockée ne dépasse le seuil, la fonction retourne `[]` sans exception.

Le seuil de 0,5 est un compromis : trop bas, le RAG retombe dans le biais inter-fiches ; trop haut, il ne se déclenche presque jamais. Le seuil est exposé en paramètre, donc ajustable empiriquement.

5.8 Justification des choix techniques

5.8.1 Sources gratuites uniquement (DuckDuckGo, GitHub, Stack Overflow)

Besoin. Sourcer des candidats sans coût d’exploitation, dans un cadre TER sans budget commercial.

Alternatives. L’API LinkedIn officielle requiert une approbation peu accessible et ses conditions d’utilisation prohibent le recrutement automatisé : *ban risk* et exposition juridique. Malt impose un contrat commercial pour l’accès programmatique. Indeed et Glassdoor sont fermés aux tiers ; un *scraping* entraînerait une violation des CGU et des risques RGPD significatifs.

Décision. DuckDuckGo (via la bibliothèque `duckduckgo-search`, libre et sans authentification), GitHub Search Users API (publique, gratuite, dédiée aux profils) et Stack Exchange API (publique, orientée tags techniques). Ces trois sources couvrent une part substantielle des profils en ligne sans exposition juridique ni coût.

Tradeoffs. DuckDuckGo n’offre pas les filtres avancés de LinkedIn et remonte du bruit d’agrégateurs (atténué par A3c). GitHub est puissant pour les profils techniques mais limité aux développeurs ayant un compte public. Stack Overflow privilégie les développeurs expérimentés. La complémentarité des trois canaux absorbe ces limitations.

5.8.2 Appels d’outils déterministes plutôt que ToolNode/bind_tools

Besoin. Le système doit être explicable et reproductible : chaque agent a une responsabilité stable, et le pipeline de recrutement doit pouvoir être suivi nœud par nœud dans LangGraph.

Alternatives. Une architecture de type ReAct aurait consisté à exposer les recherches web, le *scraping* et le RAG comme outils LangChain via `bind_tools`, puis à laisser un LLM choisir dynamiquement quels outils appeler à chaque étape. LangGraph propose aussi ToolNode pour exécuter ce type d’appels.

Décision. Le projet conserve des fonctions outils déterministes appelées explicitement par les nœuds LangGraph : A3b appelle les fonctions de recherche, A3c appelle le *scraper*, A4 lit la mémoire RAG, A8 écrit en ChromaDB. Les wrappers `@tool` restent disponibles dans `src/tools/search.py` et `src/tools/scraping.py`, mais ils ne constituent pas le mécanisme principal d’orchestration.

Justification. Dans ce cas d’usage, la séquence d’actions est connue à l’avance : analyser, générer des requêtes, collecter, filtrer, dédupliquer, évaluer, vérifier, éventuellement contacter. Confier cette séquence à un agent ReAct ajouterait de la variabilité sans bénéfice fonctionnel.

majeur. L'approche retenue réduit les coûts en *tokens*, évite les boucles ou appels d'outils inutiles, simplifie les tests et rend les responsabilités auditables dans `src/graph.py`.

Tradeoffs. Le système est moins flexible qu'un agent autonome capable d'inventer un nouveau plan d'action. Ce compromis est volontaire : pour un démonstrateur TER centré sur l'architecture multi-agents, la lisibilité, le contrôle et la traçabilité priment sur l'autonomie maximale.

5.8.3 Filtrage algorithmique plutôt que LLM

Voir section précédente. Argumentaire synthétique : coût linéaire sans appel LLM, déterminisme et auditabilité, stabilité face aux variations.

5.8.4 Décomposition A3 en trois sous-agents

Besoin. Clarifier les responsabilités du *sourcing* pour faciliter le débogage, la substitution et la compréhension architecturale.

Alternatives. Un agent A3 monolithique, plus simple à implémenter mais mêlant cognition, logistique et décision ; ou trois scripts isolés en dehors du graphe LangGraph, qui perdraient les *checkpoints* et l'observabilité unifiée.

Décision. Trois nœuds LangGraph distincts formant un pipeline linéaire. Chaque nœud a une entrée et une sortie clairement définies, un *logger* dédié et des métriques reportées à `get_metrics()`.

Tradeoffs. Trois passages dans le *runtime* LangGraph ; surcoût négligeable comparé aux appels réseau et au *scraping*.

5.8.5 *Embeddings* locaux (SentenceTransformers all-MiniLM-L6-v2)

Besoin. Produire des *embeddings* cohérents, déterministes et disponibles hors ligne.

Alternatives. `text-embedding-3-large` d'OpenAI (qualité supérieure mais clé API, coût et exposition réseau) ; Hugging Face Inference API (gratuité limitée).

Décision. `SentenceTransformer all-MiniLM-L6-v2` (384 dimensions, 80 Mo, latence d'environ 50 ms par *embedding* sur CPU). Avantages : déploiement Docker indépendant du réseau, coût marginal nul, déterminisme, taille adaptée aux recherches cosinus dans ChromaDB.

Tradeoffs. Qualité sémantique légèrement inférieure aux modèles à 3 072 dimensions ; suffisante pour discriminer des fiches au seuil 0,5. La couche `MemoireRAG` isole le modèle, qui peut être substitué sans modification des appelants.

5.8.6 ChromaDB local plutôt que service externe

Besoin. Stocker et interroger des *embeddings* avec filtrage par métadonnées (`fiche_id`).

Alternatives. Pinecone (cloud, dépendance réseau, quota gratuit limité); Weaviate (puissant mais plus lourd); Qdrant (très bon, mais surcoût opérationnel).

Décision. ChromaDB en client persistant sur disque (`./data/chromadb`). Justifications : déploiement Docker cohérent, données locales, latence négligeable, simplicité pour un prototype TER.

Tradeoffs. ChromaDB n'est pas optimisé pour les très grandes échelles (10 M+ vecteurs); non bloquant à notre échelle. La couche `MemoireRAG` encapsule les appels, rendant une migration ultérieure straightforward.

5.8.7 RAG calibré plutôt que RAG global naïf

Besoin. Améliorer la calibration des évaluations en s'appuyant sur des candidats évalués précédemment, sans introduire de biais inter-fiches.

Alternatives. RAG global (biais inter-fiches structurel); *fine-tuning* du LLM (coût et risque d'*overfitting* disproportionnés à un TER); index manuel *few-shot* par fiche (fragile).

Décision. RAG calibré par fiche (deux collections, filtrage par `fiche_id`). Élimine le biais structurel sans surcoût algorithmique notable, garantit un silence explicite sur les fiches inédites, propriétés testables.

Tradeoffs. Apport nul au premier *run*; propriété recherchée plutôt que défaut.

5.9 Extraits commentés

```
1 def hash_fiche(texte: str) -> str:
2     """Hash stable -- meme texte au whitespace pres = meme id."""
3     normalise = " ".join(texte.split()).lower()
4     return hashlib.sha1(normalise.encode("utf-8")).hexdigest()[:12]
```

Listing 5.5 – Hash déterministe de la fiche de poste (`src/tools/rag.py`)

Ce hash garantit l'idempotence : appeler deux fois `ajouter_fiche_poste()` avec le même texte ne crée pas de doublon. La normalisation rend le hash robuste aux variations de formatage.

```
1 # Etape 1 : pre-filtre (URL + mots-cles snippet)
2 pre_filtered: list[dict] = []
```

```

3 for h in hits:
4     if _is_noise_url(h.get("url", "")):
5         continue
6     snippet = f"{h.get('title', '')} {h.get('body', '')}"
7     if _is_noise_text(snippet):
8         continue
9     pre_filtered.append(h)
10
11 # Etape 2 : scraping + post-filtre
12 for i, h in enumerate(a_scraper, 1):
13     scraped = extraire_page_web_raw(h["url"])
14     if _is_noise_text(scraped):
15         n_post_drop += 1
16         continue
17     candidats.append(Candidat(...))

```

Listing 5.6 – Filtrage multi-étapes (src/agents/chercheur_filtre.py)

Le filtre applique deux tests de bruit (avant et après *scraping*) afin de capturer le bruit avant tout appel réseau et de détecter les offres déguisées révélées par leur contenu complet.

```

1 def _fiches_similaires_ids(
2     self, fiche_texte: str, seuil: float, n_max: int = 3
3 ) -> list[str]:
4     """Retourne les fiche_ids au-dessus du seuil."""
5     if self._fiches.count() == 0:
6         return []
7     results = self._fiches.query(
8         query_texts=[fiche_texte[:3000]],
9         n_results=min(n_max, self._fiches.count()),
10        include=["distances"],
11    )
12    ids = results.get("ids", [[]])[0]
13    distances = results.get("distances", [[]])[0]
14    retenus = [
15        fid for fid, dist in zip(ids, distances)
16        if (1 - dist) >= seuil
17    ]
18    return retenus
19
20 def rechercher_similaires(
21     self,
22     profil_brut: str,
23     fiche_poste: str | None = None,
24     n_results: int = 3,
25     seuil_fiche: float = _SEUIL_FICHE_DEFAUT,
26 ) -> list[dict]:
27     """Recherche calibrée si fiche_poste est fourni."""
28     if fiche_poste is not None:
29         fiche_ids = self._fiches_similaires_ids(fiche_poste, seuil_fiche)
30         if not fiche_ids:
31             return [] # silence si aucune fiche comparable

```

```

32     where_clause = {"fiche_id": {"$in": fiche_ids}}
33
34     results = self._candidats.query(
35         query_texts=[profil_brut[:2000]],
36         n_results=n_results,
37         include=["metadatas", "distances"],
38         where=where_clause if fiche_poste else None,
39     )

```

Listing 5.7 – Recherche RAG calibrée par fiche (src/tools/rag.py)

Cette implémentation illustre la logique en deux phases : d’abord filtrer les fiches par similarité seuillée, ensuite restreindre la recherche de candidats aux fiches retenues. Si aucune fiche n’atteint le seuil, la fonction retourne immédiatement une liste vide, écartant le biais inter-fiches.

```

1  def strategie_node(state: GraphState) -> dict:
2      """Genere le plan de recherche a partir du profil de competences."""
3      profil = state["profil_compétences"]
4      llm = init_chat_model(OLLAMA_MODEL, ...)
5
6      recherche_msg = (
7          f"Hard skills: {'', '.join(profil.get('hard_skills', []))}\n"
8          f"Niveau attendu: {profil.get('niveau_seniorite', 'indifferent')}\n"
9      )
10
11     response = llm.invoke([
12         SystemMessage(content=_STRATEGIE_PROMPT),
13         HumanMessage(content=recherche_msg),
14     ])
15     requetes = json.loads(response.content.strip())
16
17     # Post-traitement pour forcer le respect du niveau
18     if niveau in ("alternant", "stagiaire"):
19         exclusions = '-senior' -lead' -confirme'
20         for key in ("queries_generales", "queries_linkedin"):
21             requetes[key] = [
22                 f"{q} {mots_niveau} {exclusions}" for q in requetes.get(key,
23                 [])

```

Listing 5.8 – Génération des requêtes par A3a (src/agents/chercheur_strategie.py)

Le stratégie combine la générativité du LLM (produire des requêtes adaptées) avec un filtrage *post hoc* pour garantir le respect du niveau d’expérience. Cette hybridation équilibre flexibilité et robustesse.

```

1  seen: set[str] = set()
2  deduped: list[dict] = []
3  for h in all_hits:
4      url = h.get("url", "")
5      if not url or url in seen:
6          continue

```



```

7     seen.add(url)
8     deduped.append(h)
9
10    n_bruts = len(all_hits)
11    n_dedup = len(deduped)
12    _log.info(
13        "%d resultats apres dedup URL (%d doublons supprime).",
14        n_dedup, n_bruts - n_dedup,
15    )

```

Listing 5.9 – Déduplication par URL après agrégation (`src/agents/chercheur_collecteur.py`)

La déduplication par URL est essentielle : DuckDuckGo et l'API GitHub peuvent retourner le même profil. L'étape précède le *scraping*, ce qui évite des appels réseau redondants. Les métriques journalisées rendent compte du taux de redondance inter-sources.

5.10 Architecture API Gateway

L'API Gateway expose le système multi-agents via REST, sans jamais exposer le graphe Lang-Graph directement à l'interface graphique. Implémentée avec FastAPI sur Uvicorn (`src/api.py`), elle suit un patron de découplage strict : l'interface NiceGUI ne dialogue qu'avec des points d'accès HTTP, jamais avec le graphe. Cette séparation produit quatre bénéfices.

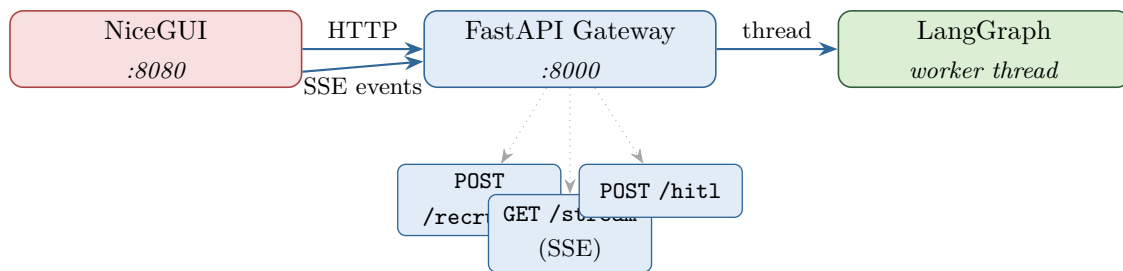
D'abord, l'**indépendance des cycles de développement**. Les membres de l'équipe peuvent se répartir le *back-end* (graphe et agents) et l'interface, sans dépendance circulaire entre les deux paquets Python.

Ensuite, la **testabilité** : `tests/test_api.py` (environ 22 cas) utilise `TestClient` de FastAPI et des *mocks* pour valider les réponses HTTP sans démarrer aucun agent. Chaque endpoint (`/recruter`, `/rapport`, `/hitl`, `/runs/{id}/stream`) est couvert en isolation.

Troisièmement, la **flexibilité de déploiement**. Le *back-end* LLM peut basculer (Ollama vers OpenAI) ou la topologie d'agents évoluer sans régénération de la couche présentation. Le fichier `docker-compose.yml` déclare API et UI comme deux services indépendants : `sma-api` peut redémarrer sans déconnecter l'utilisateur côté UI au-delà du *retry* SSE.

Enfin, l'**instrumentation décentralisée**. Les métriques et journaux du pipeline (voir section observabilité) sont collectés en mémoire de l'API, exportés en JSON dans `logs/metriques_*.json` et consultables via `GET /metrics`, sans dépendance à une base externe.

Les endpoints principaux sont : `POST /recruter` (lance un *run* en thread de fond via `BackgroundTasks`), `GET /rapport/{run_id}` (état et résultats), `GET /runs/{run_id}/stream` (flux SSE temps réel) et `POST /runs/{run_id}/hitl` (validation humaine). Chaque *run* est identifié par un UUID v4 et stocké dans un dictionnaire en mémoire, suffisant pour le démonstrateur. La figure 5.4 schématise ces interactions.



Pont synchrone → asynchrone : `queue.Queue thread-safe`.
 Le *worker* pousse des événements typés (`run_started`,
`node_completed`, `awaiting_hitl`, `run_done`); le générateur SSE les
 consomme par sondage non bloquant.

FIGURE 5.4 – Architecture API Gateway. L’UI NiceGUI ne dialogue qu’avec l’API REST, jamais avec le graphe LangGraph. Le streaming temps réel utilise des Server-Sent Events; le HITL passe par un endpoint POST distinct.

5.11 Streaming temps réel via SSE

Le modèle d’exécution du pipeline impose une synchronisation délicate : LangGraph s’exécute de manière synchrone dans un thread, tandis que FastAPI sert les réponses HTTP de manière asynchrone. Le choix de *Server-Sent Events* plutôt qu’un *WebSocket* bidirectionnel se justifie par trois éléments.

Unidirectionnalité du flux. Le pipeline pousse vers l’UI (événements de progression); les décisions HITL remontent par un endpoint POST distinct (`/hitl`). Aucun besoin d’un canal bidirectionnel persistant.

Standard HTTP/1.1 sans dépendance. SSE est consommé nativement par les navigateurs via `EventSource` et reste trivial à parser côté serveur Python. Le module `src/ui/api_client.py` implémente un parser pur (`parse_sse_chunks`) testable hors du contexte HTTP.

Latence tolérée. Les événements (fin de nœud, passage en HITL, erreur) se produisent à l’échelle de la seconde voire de la dizaine de secondes. Un sondage non bloquant à 100 ms côté serveur suffit, sans surcharger CPU ni bande passante.

L’implémentation utilise une `queue.Queue thread-safe` comme pont entre le *worker* LangGraph et le générateur SSE. Le *worker* pousse des événements typés (`run_started`, `node_completed`, `awaiting_hitl`, `run_done`, `run_error`); le générateur les consomme et les sérialise en JSON. Une sentinelle `None` marque la fin du flux et ferme proprement la connexion.

```
1 _push_event(run_id, "run_done", {
2     "rapport_chars": len(rapport),
3     "candidats": len(candidats),
4 })
```

Listing 5.10 – Émission d'un événement typé (`src/api.py`)

Le `StreamingResponse` retourné côté FastAPI ajoute l'en-tête `X-Accel-Buffering: no`, évitant le *buffering* d'un éventuel *reverse-proxy* en amont.

5.12 Human-in-the-loop : signalement par `threading.Event`

L'interruption avant le nœud A7 repose sur le mécanisme `interrupt_before` de LangGraph. Pour reprendre l'exécution après validation, il faut synchroniser deux univers : le *worker* thread (LangGraph synchrone) et le *handler* HTTP (asynchrone). Le choix de `threading.Event` plutôt qu'un mécanisme *ad hoc* se justifie par quatre propriétés.

1. **Clarté sémantique.** Un `Event` modélise exactement le problème : un signal d'un thread à un autre. Le code reste lisible : un `wait()` bloquant côté *worker*, un `set()` côté *handler*.
2. **Sécurité.** `Event.set()` est atomique en CPython ; aucun risque de course entre l'attente et la signalisation.
3. **Timeout intégré.** `wait(timeout=600)` libère le *worker* après dix minutes, évitant la fuite de processus en cas d'oubli humain.
4. **Coût minimal.** Pas de *polling* CPU, pas de file supplémentaire : un seul octet de signalisation par thread.

```

1 hitl_event: threading.Event = _runs[run_id]["_hitl_event"]
2 got_decision = hitl_event.wait(timeout=_HITL_TIMEOUT)
3 if not got_decision:
4     raise TimeoutError(
5         f"Aucune décision HITL recue en {_HITL_TIMEOUT}s..."
6     )
7 decision = _runs[run_id]["_hitl_decision"]

```

Listing 5.11 – Attente bloquante du worker sur la décision HITL (`src/api.py`)

La séquence complète, schématisée figure 5.5, est la suivante : (i) le graphe atteint l'*interrupt* et collecte les candidats en attente ; (ii) le *worker* pousse un événement `awaiting_hitl` via SSE ; (iii) l'UI affiche un formulaire à trois actions (*approve*, *skip*, *edit*) ; (iv) le recruteur valide via `POST /hitl` ; (v) l'endpoint stocke la décision et appelle `event.set()` ; (vi) le *worker* se réveille, applique la décision via `app_graph.update_state()` puis relance le streaming.

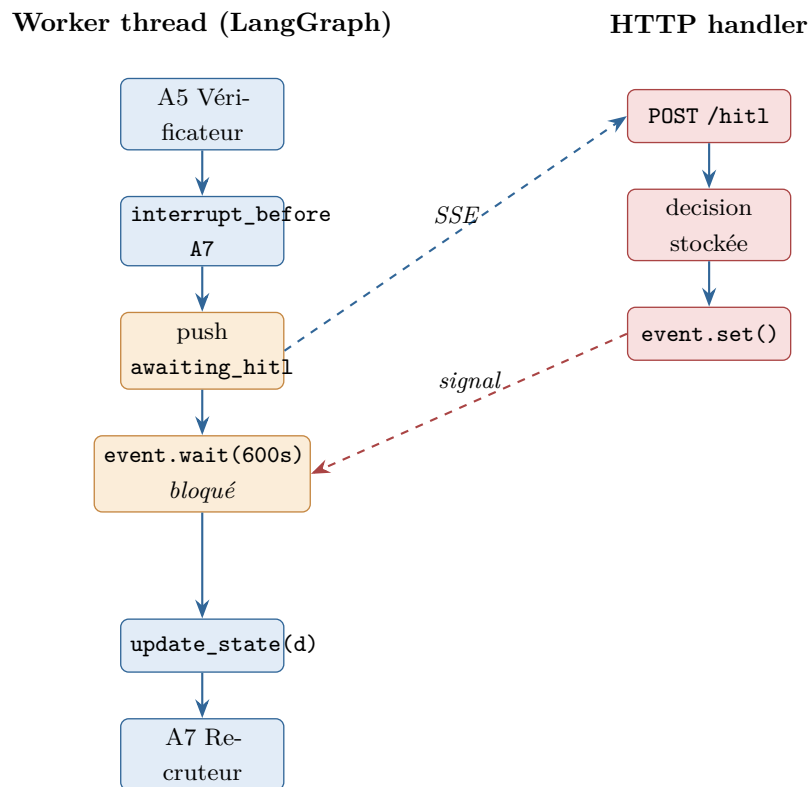


FIGURE 5.5 – Séquence du *human-in-the-loop*. Le *worker* LangGraph pousse un événement SSE puis bloque sur `Event.wait()` ; le handler HTTP traite la décision puis appelle `Event.set()`. La sémantique est claire, le *timeout* est intégré, et il n’y a pas de *polling* CPU.

5.13 Interface graphique NiceGUI

NiceGUI a été retenu après comparaison avec Streamlit, Gradio, Reflex et un *stack* JavaScript (Next.js, Vite/React). Cinq critères ont été décisifs.

1. **Langage unifié.** L’équipe est composée de trois étudiants Python ; imposer React ou Next.js réduirait la capacité de contribution à un seul membre.
2. **Multi-pages avec navigation.** NiceGUI propose des pages décorées (`@ui.page`) et un *drawer* latéral (`src/ui/layout.py`). Streamlit impose une navigation par `st.selectbox` maladroite ; Gradio ne supporte pas la pagination.
3. **Asynchrone natif.** NiceGUI s’appuie sur Quasar/Vue.js et permet des *callbacks* `async`, du SSE et des tâches d’arrière-plan sans *boilerplate*. Streamlit ré-exécute le script entier à chaque interaction, modèle inadapté à un pipeline long avec HITL.
4. **Composants riches.** Markdown, tables, formulaires, `ui.mermaid()` pour la visualisation du graphe. Reflex requiert JSX/TypeScript pour le rendu personnalisé.
5. **Déploiement léger.** Un seul processus Python, pas de serveur Node ; intégration triviale dans le `docker-compose.yml` existant.

La topologie de l’UI compte cinq pages :

- / – formulaire de lancement, statut de l’API ;

- `/runs` – historique paginé ;
- `/runs/{run_id}` – suivi en direct (Mermaid mis à jour par SSE, *timeline* d'événements, encart HITL) ;
- `/rag` – recherche de profils similaires dans ChromaDB ;
- `/metriques` – tableau d'observabilité.

Le composant critique est la consommation du flux SSE. Une coroutine `consume_stream()` itère sur les événements, met à jour le diagramme Mermaid et la *timeline* sans bloquer l'interaction utilisateur.

```

1 async def consume_stream() -> None:
2     async for event in stream_events(run_id):
3         event_type = event.get("type", "unknown")
4         data = event.get("data", {})
5         add_event(event_type, data)
6
7         if event_type == "node_completed":
8             completed.add(data.get("node", ""))
9             diagram.content = _build_mermaid(completed, ...)
10            diagram.update()
11
12 background_tasks.create(consume_stream())

```

Listing 5.12 – Consommation SSE asynchrone côté UI (`src/ui/app.py`)

Le parser SSE (`parse_sse_chunks`) est volontairement pur : il transforme un bloc de lignes en liste d'événements typés sans dépendance réseau. Cette séparation rend le parser testable indépendamment.

5.14 Observabilité et tests

5.14.1 Collecte et export de métriques

Le module `src/observabilite.py` expose un singleton `PipelineMetrics`. Chaque nœud invoque `get_metrics().fin("nom_etape", **kwargs)` avec les métriques métier qui le concernent.

```

1 m = get_metrics()
2 m.fin("A3c_filtre", n_entree=30, n_sortie=8, taux_bruit=0.73)

```

Listing 5.13 – Annotation d'une étape avec ses métriques métier

À la fin du pipeline, le nœud rapport invoque `m.exporter()`, qui sérialise les résultats dans `logs/metriques_AAAAMMJJ_HHMMSS.json`. L'historique est consultable via `GET /metrics` et affiché dans la page `/metriques` de l'UI.

5.14.2 Journalisation structurée

Le module `src/logger.py` impose un format ISO 8601 + niveau + composant. Tous les agents et l'API utilisent `get_logger("composant")`, ce qui centralise la production sur `stdout` et la rend capturable par Docker ou `systemd`.

5.14.3 Couverture de tests

La suite (81 cas environ) couvre cinq domaines :

Fichier	Cas	Périmètre
<code>test_api.py</code>	22	Endpoints REST en isolation, mocks du pipeline.
<code>test_graph.py</code>	14	Topologie LangGraph, routage conditionnel.
<code>test_filtre.py</code>	18	Détection de mots-clés et de bruit.
<code>test_deduplicateur.py</code>	11	Distance Levenshtein sur noms et URLs.
<code>test_rag.py</code>	16	Insertions ChromaDB et calibration par fiche.

TABLE 5.2 – Répartition des tests automatisés.

Aucun test d'interface n'est fourni, choix assumé pour les raisons suivantes : (i) les widgets NiceGUI sont une couche de présentation déclarative ; tester un rendu nécessiterait un navigateur sans tête (Selenium ou Playwright), coûteux en CI et à maintenir ; (ii) la logique métier réside dans le *back-end* REST, déjà couvert par `test_api.py` ; (iii) le périmètre TER privilégie l'argumentation architecturale sur la couverture cosmétique de l'UI.

5.14.4 Conteneurisation et déploiement

L'ensemble est livré sous forme d'un `docker-compose.yml` déclarant trois services : `ollama` (LLM local), `sma-api` (FastAPI) et `sma-ui` (NiceGUI). Deux volumes hôtes (`./logs` et `./data`) persistent respectivement les exports d'observabilité et la base ChromaDB. La figure 5.6 en donne la topologie.

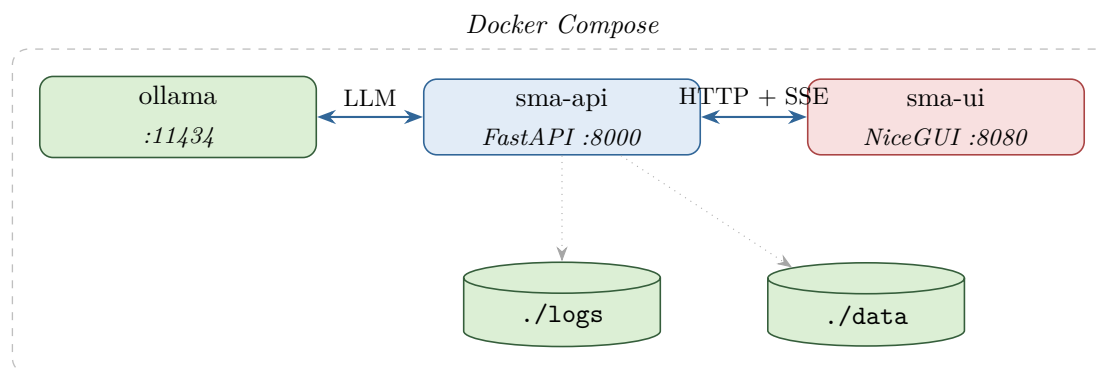


FIGURE 5.6 – Déploiement Docker Compose à trois services. Le LLM Ollama, l'API et l'UI tournent dans des conteneurs isolés. Deux volumes persistent les journaux d'observabilité (`./logs`) et la base ChromaDB (`./data`).

Cette organisation garantit la reproductibilité (une commande `docker compose up` suffit), l'isolation entre couches et la persistance des données entre redémarrages, sans dépendance à un service externe.

5.15 Justification des choix techniques

Choix	Besoin	Alternative principale	Décision et tradeoff
API Gateway REST	Découplage présentation / orchestration	UI important <code>build_graph()</code>	Gateway : plus de souplesse de déploiement, surcoût négligeable
SSE unidirectionnel	Flux pipeline → UI	WebSocket bidirectionnel	SSE : protocole standard, parsing trivial, pas de canal inverse nécessaire
<code>threading.Event</code> HITL	Synchronisation <i>worker</i> / HTTP	<i>Polling</i> ou <i>future asyncio</i>	Event : sémantique claire, <i>timeout</i> intégré, coût minimal
NiceGUI	Multi-pages temps réel en Python	Streamlit / Gradio / React	NiceGUI : Python unifié, modèle événementiel adéquat
Docker-compose 3 services	Reproductibilité, démo offline	Monolithe <i>localhost</i>	3 services : isolation, persistance volumique

TABLE 5.3 – Synthèse des choix techniques de la couche service.

5.16 Extraits commentés

```

1 @app.post("/recruter", response_model=RecruterResponse)
2 async def recruter(
3     req: RecruterRequest, background_tasks: BackgroundTasks
4 ) -> RecruterResponse:
5     run_id = str(uuid.uuid4())
6     _runs[run_id] = {
7         "run_id": run_id,
8         "fiche_poste": req.fiche_poste,
9         "status": "running",
10        "_event_queue": queue.Queue(),          # pont synchrone -> async
11        "_hitl_event": threading.Event(),       # signal de decision
12        "_with_interrupt": not req.no_interrupt,
13    }
14
15    background_tasks.add_task(
16        _run_pipeline,
17        run_id,
18        req.fiche_poste,

```

```

19     not req.no_interrupt,
20 )
21 return RecruiterResponse(run_id=run_id, status="running", ...)

```

Listing 5.14 – Initialisation d'un *run* et lancement en arrière-plan (`src/api.py`)

La file SSE et l'*Event* HITL sont initialisés dès la requête, ce qui découple le cycle de vie du *run* de celui de la requête HTTP.

```

1 def _push_event(run_id: str, event_type: str, data: dict) -> None:
2     info = _runs.get(run_id)
3     if info is None:
4         return
5     q = info.get("_event_queue")
6     if q is not None:
7         q.put({
8             "type": event_type,
9             "data": {**data, "ts": datetime.now(...).isoformat()},
10        })

```

Listing 5.15 – Capture d'événements dans la file SSE (`src/api.py`)

Chaque événement reçoit un horodatage ISO et reste sérialisable en JSON. L'absence de *run_id* en mémoire est tolérée silencieusement, évitant qu'une condition de course ne tue le *worker*.

```

1 m = get_metrics()
2 m.noter(
3     "pipeline",
4     n_profils_bruts=n_bruts,
5     n_profils_dedup=n_dedup,
6     n_scores=len(candidats_scores),
7     n_valides=len(candidats_valides),
8 )
9 metriques_export = m.exporter()
10 metriques_resume = m.resume_texte()

```

Listing 5.16 – Export des métriques en fin de pipeline (`src/agents/orchestracteur.py`)

L'export est non destructif : les métriques restent disponibles en mémoire pour les requêtes ultérieures sur `/metrics` ou pour la page `/metriques` de l'UI.

5.17 Synthèse du chapitre

L'analyse conduite dans ce chapitre montre que le système réalise effectivement la convergence SMA / Agentic AI annoncée au chapitre 2. Trois lectures croisées permettent de le constater.

Lecture SMA classique. Le système hérite de plusieurs apports formels des SMA. La topologie en graphe orienté typé (§ 5.A) joue le rôle d'un *blackboard* avec contrats explicites entre agents. Le patron de validation pair-à-pair entre A4 et A5 (§ 5.A) implémente une variante des *checks*

and balances formels. Le routage conditionnel par seuil rend l'arbitrage explicite et auditable. La mémoire vectorielle calibrée par fiche (§ 5.B) introduit une notion de contexte persistant proche de celle réclamée dans la littérature SMA. Enfin, le HITL via `interrupt_before` (§ 5.A et § 5.C) réalise la *governance and verification* préconisée pour combler le déficit d'explicabilité de l'Agentic AI.

Lecture Agentic AI. Le système exploite la flexibilité cognitive des LLM partout où elle apporte une plus-value : extraction structurée par A2, génération de requêtes adaptées par A3a, scoring multi-axes par A4, contrôle de cohérence par A5, rédaction de messages par A7. À chaque fois, le LLM est encadré par un schéma de sortie strict (JSON, Markdown), une température contrôlée, et un agent de contrôle indépendant lorsque la décision est sensible. Le RAG fournit la *persistent memory* canonique des architectures agentiques.

Lecture opérationnelle. La couche service (§ 5.C) traduit ces patrons théoriques en un système exploitable : API Gateway, streaming temps réel, interface multi-pages, déploiement Docker, observabilité JSON. Aucune décision technique n'a été prise sans la rapporter au cahier des charges du chapitre 4 et à l'argumentaire théorique du chapitre 2. Le couplage entre théorie et implémentation est ainsi explicite et défendable.

Le chapitre suivant tire les conclusions de ce travail et identifie les limites résiduelles ainsi que les perspectives d'évolution.

Chapitre 6

Conclusion et perspectives

6.1 Bilan

Ce TER avait pour ambition d’articuler deux paradigmes de l’intelligence artificielle distribuée — les Systèmes Multi-Agents classiques et l’Agentic AI — dans un cas d’étude concret. Trois objectifs avaient été énoncés au chapitre 1 : un objectif théorique (état de l’art et analyse comparative), un objectif méthodologique (sélection argumentée d’un *framework*) et un objectif applicatif (conception, implémentation, évaluation d’un SMA de recrutement).

Les trois objectifs ont été tenus.

Sur le plan théorique. Le chapitre 2 a synthétisé dix-huit études du dossier *Etudes/*, dégagé les invariants des SMA (autonomie, propriétés des agents, architectures, coordination formelle, MARL, émergence) et de l’Agentic AI (clarification conceptuelle en trois niveaux, double paradigme symbolique/neuronal, architecture modulaire), puis articulé une analyse comparative en trois temps : éléments communs, différences structurelles, apports croisés. Cette analyse a fourni un cadre interprétatif pour l’ensemble des choix techniques ultérieurs.

Sur le plan méthodologique. Le chapitre 3 a formalisé neuf critères d’évaluation, comparé neuf *frameworks* candidats, et conduit à la sélection argumentée de LangGraph. La décision n’a pas été présentée comme une évidence mais comme la conjonction explicite de cinq éléments (adéquation structurelle, convergence avec l’argumentation théorique, indépendance, maturité, expressivité argumentative).

Sur le plan applicatif. Le système comporte huit agents spécialisés (A1 à A8, dont A3 décomposé en trois sous-agents), une mémoire vectorielle calibrée par fiche (deux collections ChromaDB reliées par `fiche_id`), une API Gateway FastAPI avec streaming SSE et endpoints HITL, une interface graphique NiceGUI à six pages, une suite d’environ quatre-vingts tests automatisés, un déploiement Docker en trois services. Le code est livré avec un journal de décisions de conception (`rapportPlan.md`) qui documente *ex ante* les principaux choix.

Contrainte transversale tenue. La contrainte « chaque choix doit être justifié » a été respectée dans l’ensemble du rapport : besoin, alternatives considérées, décision, compromis assumés sont explicités pour chaque décision technique majeure (cf. chapitre 5, sections *Justification des choix*).

6.2 Limites identifiées

Plusieurs limites sont assumées et explicitement reconnues.

Qualité du LLM local. Le modèle Ollama choisi pour la confidentialité offre une qualité de raisonnement inférieure à celle des modèles de classe Claude Opus ou GPT-4. L’écart est atténué par un *prompt engineering* contraint, une température basse pour les tâches d’évaluation, une structuration JSON stricte des sorties, et un agent de contrôle indépendant (A5). Il n’est cependant pas annulé : un cas réel exigeant pourrait justifier un basculement ponctuel vers une API externe pour la phase d’évaluation, à condition d’anonymiser au préalable les données candidats.

Couverture du *sourcing*. Le périmètre des sources gratuites (DuckDuckGo, GitHub, Stack Exchange) couvre largement les profils techniques publics, mais sous-représente les métiers non techniques et les profils peu actifs en ligne. L’extension à d’autres sources (Mastodon, GitLab, Hacker News) est triviale via le patron A3a/b/c. L’absence volontaire de LinkedIn et de Malt est une décision argumentée (RGPD, conditions d’utilisation, risque juridique) et non un oubli.

Mémoire RAG limitée aux runs précédents. La calibration par fiche n’apporte une plus-value qu’à partir du second *run* sur une fiche similaire. Au premier *run*, le RAG est silencieux. C’est une propriété recherchée (ne pas injecter de signal là où il n’y en a pas) mais cela limite la calibration pour des cas isolés. Une amélioration possible serait l’injection d’un référentiel initial de candidats étiquetés (*seed*) pour amorcer la mémoire.

Absence d’apprentissage en ligne. Le système n’apprend pas des décisions humaines HITL. Si un recruteur invalide systématiquement les candidats que A5 marque *douteux*, cette information n’est pas exploitée pour ajuster A4 ou A5. Une intégration MARL ou un mécanisme de *feedback* explicite est laissé aux perspectives.

Multi-tenant non traité. Les données sont mutualisées dans une seule instance ChromaDB. Un déploiement servant plusieurs recruteurs avec des données strictement isolées exigerait une refonte de la couche persistance.

6.3 Perspectives

Vérification formelle des invariants. Plusieurs invariants du graphe (par exemple : aucun candidat ne peut être validé sans avoir été évalué) pourraient être vérifiés statiquement par des assertions sur `GraphState` ou un système de types plus riche. Cette piste rejoindrait l’apport SMA → Agentic AI identifié au chapitre 2 (gouvernance et vérification formelle).

Apprentissage par renforcement multi-agents. Une trajectoire naturelle pour étendre le système consisterait à observer les décisions HITL et les ajustements de A5, et à fine-tuner les seuils `SCORE_SEUIL_CONTACT` et `SCORE_SEUIL_VIABLE`, voire les *prompts* de A4 et A5, par un mécanisme MARL. La littérature SMA dispose d’un cadre formel pour cela [7].

Adoption du Model Context Protocol (MCP). L’écosystème agentique s’oriente vers une standardisation de la communication entre agents et outils via le MCP. Une refonte du *tool calling* de A3 et A4 pour exposer les outils comme des serveurs MCP rendrait le système interopérable avec d’autres orchestrateurs.

Communautés agentiques humain-IA. Conformément à la trajectoire évolutive identifiée par MILOSEVIC et RABHI [11], le système pourrait évoluer vers un écosystème où plusieurs recruteurs humains collaborent avec des agents IA partagés, dans un cadre de gouvernance explicite. Cela suppose une couche d’authentification, des rôles formels et une journalisation inter-utilisateurs.

Évaluation utilisateur. Au-delà des tests unitaires, une évaluation par des recruteurs réels sur des fiches de poste réelles permettrait de mesurer empiriquement la qualité du *sourcing*, la pertinence des scores et l’utilité du HITL. Une telle évaluation a été écartée du périmètre TER pour des raisons de calendrier et de protection des données, mais constitue une suite naturelle du travail.

6.4 Considérations éthiques et RGPD

Le recrutement automatisé manipule des données à caractère personnel sensibles (identité, parcours, compétences supposées). Plusieurs principes ont guidé la conception :

- **Minimisation** : seules les données nécessaires à l’évaluation sont collectées et stockées. Les profils invalides ne sont pas persistés en RAG.
- **Localité** : le LLM s’exécute sur la machine du déploiement (Ollama) ; aucune donnée candidat n’est envoyée à un fournisseur tiers. Les sources de *sourcing* sont publiques.
- **Souveraineté décisionnelle** : aucune action irréversible (envoi de message) n’est effectuée sans validation humaine explicite. Le système assiste le recruteur, ne le remplace pas.
- **Transparence du scoring** : chaque score est accompagné d’explications structurées par axes ; A5 peut invalider ou ajuster ; le recruteur peut éditer la liste finale.

- **Auditabilité** : chaque exécution est *checkpointée*, journalisée, exportable en JSON. Une demande de droit d'accès ou de droit à l'oubli peut être satisfaite.
- **Hors-scope LinkedIn et Malt** : le scraping de plateformes interdisant explicitement la collecte automatisée a été écarté pour des raisons à la fois juridiques et éthiques.

Un usage en production exigerait néanmoins une analyse d'impact relative à la protection des données (AIPD) formelle, une politique de durée de conservation explicite, et l'information préalable des candidats sourcés. Ces éléments dépassent le périmètre d'un TER mais constituent un prérequis indispensable à toute industrialisation.

Le code source, les tests, les exports d'observabilité et le journal de décisions de conception (`rapportPlan.md`) accompagnent ce rapport et peuvent être consultés pour reproduire l'analyse.

Bibliographie

- [1] Ali DORRI, Salil S. KANHERE et Raja JURDAK. « Multi-Agent Systems : A Survey ». In : *IEEE Access* 6 (2018). Etude SMA 08 du dossier de recherche., p. 28573-28593.
- [2] General SURVEY. *General Survey on Multi-Agent Theory and Applications*. Etude SMA 04. 2023.
- [3] SAPKOTA et al. *AI Agents vs Agentic AI*. Etude Agentic AI 04. 2026.
- [4] Abou ALI et al. *A Comprehensive Survey on Agentic AI*. Etude Agentic AI 02. 2026.
- [5] J. MALDONADO et al. *Systèmes multiagents : architectures et propriétés*. Etude SMA 02 et 07. 2024.
- [6] FIPA. *FIPA Agent Communication Language Specifications*. <http://www.fipa.org/repository/aclspecs.html>. 2002.
- [7] CMU. *Multi-Agent Reinforcement Learning : a Machine Learning Perspective*. Etude SMA 05. 2022.
- [8] Visual Survey of AGENT-BASED COMPUTING. *A Visual Survey of Agent-Based Computing*. Etude SMA 03. 2023.
- [9] WJARR. *Future of Distributed Artificial Intelligence and Multi-Agent Systems*. Etude SMA 06. 2024.
- [10] SCHNEIDER. *From Generative to Agentic : a paradigm shift*. Etude Agentic AI 10. 2025.
- [11] MILOSEVIC et RABHI. *Agentic Communities : hybrid human-AI ecosystems*. Etude Agentic AI 09. 2026.
- [12] PATI. *Agentic AI : Technologies and Societal Implications*. Etude Agentic AI 06. 2025.
- [13] WAVESTONE. *Agentic AI Playbook*. Etude Agentic AI 01. 2024.
- [14] ALVA et PANDEY. *Age of Generative Models : the rise of Agentic Systems*. Etude Agentic AI 03 et 10. 2025.
- [15] VAIDHYANATHAN et TAIBI. *A Critical Analysis of Agentic AI Frameworks*. Etude Agentic AI 07. 2026.
- [16] HUGHES et al. *AI Agents and Agentic Systems : a structured review*. Etude Agentic AI 11. 2025.
- [17] TOSEM. *A Year in TOSEM : Agentic Software Engineering*. Etude Agentic AI 08. 2025.

- [18] PATEL et al. *A Systematic Review of Generative AI for Agentic Systems*. Etude Agentic AI 05. 2025.
- [19] LANGCHAIN INC. *LangGraph : a low-level orchestration framework for stateful agents*. <https://langchain-ai.github.io/langgraph/>. 2024.
- [20] LANGCHAIN INC. *LangChain Documentation*. <https://python.langchain.com/>. 2024.
- [21] MICROSOFT RESEARCH. *AutoGen : Multi-Agent Conversation Framework*. <https://microsoft.github.io/autogen/>. 2024.
- [22] CREWAI. *CrewAI : Framework for orchestrating role-playing autonomous AI agents*. <https://www.crewai.com/>. 2024.
- [23] LLAMAINDEX. *LlamaIndex : Data framework for LLM applications*. <https://www.llamaindex.ai/>. 2024.
- [24] OLLAMA. *Ollama : run large language models locally*. <https://ollama.com/>. 2024.
- [25] Nils REIMERS et Iryna GUREVYCH. *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*. <https://www.sbert.net/>. 2019.
- [26] CHROMA. *ChromaDB : the AI-native open-source embedding database*. <https://www.trychroma.com/>. 2024.
- [27] Sebastián RAMÍREZ. *FastAPI : Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com/>. 2024.
- [28] W3C. *Server-Sent Events Specification*. <https://www.w3.org/TR/eventsource/>. 2015.
- [29] ZAUBERZEUG. *NiceGUI : a Python web UI framework based on Quasar*. <https://nicegui.io/>. 2024.